TECHNICAL DOCUMENTATION

VANTA

Virtualized Adversarial Network & Technical Assessment

"Don't give the contestant a problem. Give them a world."

A complete guide to the project: every concept, every feature, every file, the HTTP API, the database, the C protocols, testing, operations, limitations, how to extend it, and how to present it.

TRACKED FILES	LINES OF CODE	AUTOMATED TESTS	DEFECT CLASSES
86	7,404	35	2

Contents

1 Introduction 4
- 1.1 What VANTA is 4
- 1.2 The problems VANTA was built to solve 4
- 1.3 Design principles 4
- 1.4 What is in the box 5
- 1.5 How to read this document 5

2 Core concepts 7
- 2.1 World 7
- 2.2 Archetype 7
- 2.3 Seed and determinism 8
- 2.4 Bug variant 8
- 2.5 Genes and difficulty 8
- 2.6 Pins 9
- 2.7 Hidden test harness 9
- 2.8 Sandbox and sanitizers 9
- 2.9 Reference fixes 9
- 2.10 Fitness and viability 10
- 2.11 Evolution 10
- 2.12 Probe 10
- 2.13 Witness 10
- 2.14 Rating and calibrated difficulty 10
- 2.15 Cohort 10
- 2.16 Originality 11
- 2.17 Identity 11

3 Architecture 12
- 3.1 The big picture 12
- 3.2 Technology stack 12
- 3.3 The contestant's journey 13
- 3.4 What happens on each request 13
- 3.5 Repository layout 14

4 Features in depth 15
- 4.1 World generation (archetypes) 15
- 4.2 The evolution engine 18
- 4.3 Grading submissions 19
- 4.4 Probe lab (experiments) 20
- 4.5 Witnesses 21
- 4.6 Adaptive difficulty (ratings) 23
- 4.7 Cohorts 24
- 4.8 Originality analysis 24
- 4.9 The web workbench 25
- 4.10 The Discord bot 25
- 4.11 Permissions and admin 26

5 File-by-file reference 27

5.1 Repository root 27
5.2 backend/ — the API service 27
5.3 backend/app/api/ 28
5.4 backend/app/core/ — state and bookkeeping 28
5.5 backend/app/evolution/ — the generator's search loop 30
5.6 backend/app/execution/ 31
5.7 backend/app/investigation/ — probe lab and witnesses 31
5.8 backend/app/worldgen/ — world generation 32
5.9 backend/tests/ — automated tests 33
5.10 discord_bot/ — the Discord adapter 33
5.11 frontend/ — configuration 34
5.12 frontend/src/ — application code 35
5.13 docs/ — this documentation 37
5.14 Generated and local files (not in git) 37

6 HTTP API reference 38
6.1 Conventions 38
6.2 Service and catalogue 38
6.3 Worlds 38
6.4 Investigation 39
6.5 Profiles 41
6.6 Cohorts 41
6.7 Admin 42

7 Data model 43
7.1 What "nothing is stored" means 43
7.2 Table worlds 43
7.3 Table profiles 44
7.4 Table attempts 44
7.5 Table activity 44
7.6 Table cohorts 44
7.7 Table cohort_members 45
7.8 Table cohort_submissions 45
7.9 How the tables relate 46
7.10 What is kept only in memory 46

8 C protocols and formats 47
8.1 How the two C files are put together 47
8.2 Grading output 47
8.3 Scheduler probe output (job_queue) 47
8.4 Pool probe output (pool) 48
8.5 Scenario JSON schemas 48
8.6 Code safety rules 48

9 Testing 49
9.1 Running the tests 49
9.2 test_evolution.py — the generator 49
9.3 test_investigation.py — probe lab, witnesses, ratings 49
9.4 test_cohorts.py — cohorts, originality, permissions 50
9.5 test_discord_commands.py — the Discord bot 50
9.6 Browser checks 51

9.7 Frontend type-checking 51

10 Installing and running 52

10.1 Requirements 52
10.2 First-time setup 52
10.3 Starting everything 52
10.4 Configuration (environment variables) 52
10.5 Setting up the Discord bot, step by step 53
10.6 Ports and processes 53
10.7 Common tasks 53

11 Limitations, security and future work 54

11.1 Security limits 54
11.2 Functional limits 54
11.3 Ideas for future work 55

12 Extending VANTA: adding a defect class 56

12.1 1. Write the archetype 56
12.2 2. Add two different reference fixes 56
12.3 3. Register it 57
12.4 4. Probe kit (experiments) 57
12.5 5. Witness kit 57
12.6 6. Tests 57
12.7 Adding an API endpoint or page 57

13 Presenting VANTA 58

13.1 Before you start 58
13.2 The script 58
13.3 Likely questions 58

14 Glossary 60

CHAPTER 1

Introduction

1.1 What VANTA is

VANTA stands for *Virtualized Adversarial Network & Technical Assessment*. Its motto is:

> *"Don't give the contestant a problem. Give them a world."*

VANTA is a platform for technical challenges and hackathons in the C language. Instead of handing someone a fixed exercise ("write a function that reverses a list"), VANTA builds a small, working-but-broken *system* on demand. One example is a scheduler for a shared 3D printer. Another is a booking system for parking spots. The system has a real defect hidden in it. The contestant's job is to work out what is wrong, test their theory, and submit C code that fixes it.

Three things make VANTA different from an ordinary problem set:

1. **Nothing is stored in advance.** There is no bank of problems and no answer key. Every world is generated by a program from a random number (a *seed*). The person who wrote VANTA doesn't know in advance what a given seed will produce.
2. **Every world is proven before anyone sees it.** VANTA compiles and runs each candidate world and checks three things. The planted bug really makes the hidden tests fail. The world can be fixed. And at least two *architecturally different* fixes pass. Worlds that fail any check are thrown away.
3. **Investigation without arguing with bots.** Earlier versions of the project used chat-bot characters with written backstories. Contestants ended up trying to talk the bots into revealing the answer. VANTA replaces this with *execution-grounded witnesses*. These are people whose every statement is computed by actually running the broken system. A *probe lab* lets contestants design their own experiments.

1.2 The problems VANTA was built to solve

Problem with typical challenge platforms	How VANTA answers it
Problems leak. Solutions get shared online and copied.	Every world is generated fresh. In a cohort, each person's world has different names and numbers, so copied code doesn't even compile.
One "correct" answer encourages pattern-matching.	Each world is proven to admit at least two different correct designs.
Chat-bot "characters" get argued with in circles.	Witnesses hold no facts. Their answers come from running code, so the same question always gets the same answer and there is nothing to argue about.
Beginners and experts get the same difficulty.	Each person has a rating. The next world is evolved to sit slightly above their level.
A grader that is a person or a language model can be inconsistent.	Grading is pure execution: compile, run hidden tests under sanitizers, read the results.

1.3 Design principles

These ideas show up again and again in the code. Knowing them makes every file easier to read.

- **Programs, not content.** An *archetype* is a Python class that *writes* a world (C code, tests, story) from a seed. It fixes only the *shape* of a defect, never its concrete numbers or names.
- **Prove it by running it.** Whenever VANTA claims something about a world, a compiler and a test harness back the claim up. Examples are "it's broken", "it's fixable", "this witness waited 5 days" and "these two numbers disagree".
- **Many correct answers.** The fitness function rewards worlds where several different fixes pass. Witness replays expose the *trade-offs* between correct designs.
- **Bounded interaction.** Every question gets one answer. No component ever gets into an open-ended conversation that a contestant could "win".
- **Deterministic.** The same seed (and the same settings) always produces the same world. That makes bugs reproducible and results fair.
- **One store, many views.** The web UI and the Discord bot are both clients of the same HTTP API and the same database.

1.4 What is in the box

Area	Feature	Where it lives
Generation	Two defect classes (*archetypes*), each with three bug variants and several story settings	`backend/app/worldgen/`
Generation	Evolutionary search: generate → compile → test → score → select → mutate	`backend/app/evolution/`
Grading	C sandbox with a static safety check, sanitizers and timeouts; hidden-test harness protocol	`backend/app/execution/`, `backend/app/worldgen/evaluate.py`
Investigation	Probe lab: contestant-written experiments run against the original system or their own code	`backend/app/investigation/probes.py`
Investigation	Witnesses who lived through one real run, can be asked bounded questions, and can relive the same period under the contestant's code	`backend/app/investigation/witnesses.py`
Adaptation	Chess-style (Elo) rating per person and calibrated difficulty for the next world	`backend/app/core/profiles.py`
Groups	Cohorts: one defect for a whole room, a unique world per member, a host console	`backend/app/core/cohorts.py`
Groups	Originality analysis: how many genuinely different approaches were accepted	`backend/app/core/originality.py`
Interfaces	Web workbench (React + TypeScript)	`frontend/`
Interfaces	Discord bot with 16 slash commands	`discord_bot/`
Quality	35 automated tests covering generation, investigation, cohorts and Discord	`backend/tests/`

1.5 How to read this document

- **Chapter 2 (Concepts)** explains every idea in plain language. Read it first if you are new.
- **Chapter 3 (Architecture)** shows how the pieces connect and what happens on each request.
- **Chapter 4 (Features)** goes deep on every feature, including the exact numbers the code uses.

- **Chapter 5 (File reference)** walks through *every file in the repository*, one by one.
- **Chapters 6–8** are references: the HTTP API, the database tables, and the text protocols the C programs use to report results.
- **Chapters 9–14** cover testing, running the project, limitations, how to extend it, a demo script for presenting it, and a glossary.

Code identifiers are written in `monospace`. File paths are relative to the repository root (the folder that contains `backend/`, `frontend/` and `discord_bot/`).

CHAPTER 2

Core concepts

This chapter explains every idea VANTA is built from, in the order you meet them. Each concept has a short definition, an example, and the file where it lives.

2.1 World

A **world** is one complete challenge. It contains:

- a **story**: a title (for example *University Library*), a briefing, and a task description;
- a **shared header**: C declarations (a `struct` and function prototypes) that both the contestant's code and the test program see;
- **starter source**: the current, broken C code the contestant starts from;
- **requirements**: a list of properties a correct solution must satisfy, written in plain English;
- a **hidden test harness**: a C program with a `main()` that calls the contestant's functions and checks the requirements;
- **ground truth**: which bug was planted and which constants were used. Only admins can see this.

Every world has an 8-character `world_id` (for example `66614ce6`) and the `seed` it was grown from. In code it is the `GeneratedWorld` dataclass (`backend/app/worldgen/archetype.py`). A stored world is a `WorldRecord` (`backend/app/core/world_manager.py`).

Here is the shared header and starter source of a real generated world. It is a library locker pool with the *off-by-one* bug:

```c
#define MAX_POOL 128
struct Locker {
    int inUse;
    int holderId;
};
int acquireSlot(struct Locker pool[], int poolSize, int holderId);
int returnItem(struct Locker pool[], int poolSize, int holderId);

int returnItem(struct Locker pool[], int poolSize, int holderId) {
    int idx = holderId % (poolSize + 1);
    if (idx >= 0 && idx < poolSize && pool[idx].inUse) { pool[idx].inUse = 0; return 1; }
    return 0;
}
```

The contestant sees the code, the story and the requirements. They don't see the comment that would say what is wrong (there isn't one), the hidden tests, or the name of the bug.

2.2 Archetype

An **archetype** is a *program that writes worlds*. It is a Python class with one important method:

```
generate(seed, genes=None, pins=None) -> GeneratedWorld
```

An archetype decides the *shape* of a defect, for example "a scheduler that can starve long jobs". Everything concrete is drawn from a random number generator seeded with the seed: the struct name, function names, story setting, numbers, and which of several bug *variants* to plant. VANTA currently has two archetypes:

Archetype id	Title	What breaks
`starvation_fairness`	Shared Resource Fairness	A scheduler for one shared machine (3D printer, washing machine, GPU…) picks the wrong job next, so some jobs wait forever or the machine stalls
`pool_leak`	Resource Pool Integrity	A fixed pool of slots (lockers, parking spots, loaner laptops…) loses track of what is checked out, so capacity leaks away

Files: `backend/app/worldgen/archetype.py` (the base class), `backend/app/worldgen/archetypes/starvation.py`, `backend/app/worldgen/archetypes/pool_leak.py`.

2.3 Seed and determinism

A **seed** is an integer. Every random choice in VANTA comes from a random number generator created from a string such as `"pool_leak:42"`. So the same seed always gives the same world. This matters for three reasons. A bug report can be reproduced exactly. A cohort can give every member a different but repeatable world. And the evolution loop's results can be tested (`test_same_seed_reproduces_same_world`).

2.4 Bug variant

Each archetype can plant one of several **variants** of its defect. They are different mistakes that lead to the same kind of failure:

Archetype	Variant	The mistake in the starter code
`starvation_fairness`	`no_close_check`	Serves the oldest job but ignores how many minutes are left before closing, so it picks jobs the machine can't finish today and the machine stalls
`starvation_fairness`	`pure_shortest_first`	Always serves the shortest job, so a long job can wait forever behind a stream of short ones
`starvation_fairness`	`truncating_priority`	Uses an integer-division score `minutesNeeded / waited` that collapses to 0, so the ordering stops meaning anything
`pool_leak`	`linear_search_first_match`	`release` matches a stale `holderId` on a slot that is already free, so a second release "succeeds"
`pool_leak`	`off_by_one_index`	`release` computes the slot as `holderId % (poolSize + 1)` and frees the wrong slot or none
`pool_leak`	`no_double_checkout_guard`	`checkout` lets one holder take a second slot, which is never given back

2.5 Genes and difficulty

Genes are six numbers between 0 and 1 that control *how hard* a world is. The evolution layer turns these dials. Archetypes read the ones they understand.

Gene	Meaning	Used by
`system_size`	How much code and how many components to read	(reserved)
`bug_subtlety`	How hidden the defect is	starvation: picks subtler variants more often

Gene	Meaning	Used by
`constant_magnitude`	How big the numbers are (day length, pool size)	both
`noise`	Irrelevant or decoy material	(reserved)
`edge_density`	How many edge cases the hidden tests probe	pool: number of churn cycles
`reasoning_depth`	How long the chain is from symptom to cause	(reserved)

A single **target difficulty** between 0 and 1 is mapped onto all six genes (`EvolutionEngine._target_vector`). Contestants choose *Gentle* (0.2), *Standard* (0.45), *Hard* (0.7) or *Brutal* (0.9), or let VANTA calibrate it to their rating.

2.6 Pins

Pins fix some of an archetype's choices that would otherwise be random. Currently a pin can lock the `variant` and the story `domain` (setting). A cohort pins both, so every member faces the same bug in the same setting. Names and numbers are still drawn from each member's own seed.

2.7 Hidden test harness

The **harness** is a C `main()` program generated alongside the world. It calls the contestant's function in many scenarios and prints one line per test:

```
VANTA_TEST_RESULT|hidden_starvation_stream|1|0
VANTA_OVERALL|0
```

The line format is `name | violation bitmask | passed (1/0)` . The Python side reads these lines (`backend/app/worldgen/evaluate.py`). Contestants see public checks by name and hidden checks only as "hidden check 3". A name like `hidden_starvation_stream` would give the scenario away.

2.8 Sandbox and sanitizers

The **C sandbox** (`backend/app/execution/c_sandbox.py`) compiles the harness and the contestant's file together with `gcc` and runs the result with a time limit. Two modes exist:

- **sanitized** (for contestants): `-Wall -Wextra -fsanitize=undefined,address -O0 -g` . Undefined behaviour, buffer overflows and use of invalid memory become crashes instead of silent luck.
- **fast** (for the internal evolution loop): `-w -O1` , no sanitizers, for speed.

Before compiling, a **static safety check** rejects code containing dangerous calls such as `system(` , `fork(` , `fopen(` or `#include <unistd.h>` .

2.9 Reference fixes

Reference fixes are known-good solutions written into VANTA (`backend/app/evolution/reference_fixes.py`). Each archetype has two that differ in *approach*. For the scheduler, one uses an "aging guard" and the other "oldest first". For the pool, one scans forward and the other backward. They are never shown to contestants. They exist only so the evolution loop can *prove* a world is fixable in more than one way.

2.10 Fitness and viability

When the evolution loop builds a candidate world, it measures its **fitness**:

- **broken**: the starter fails at least one of its own hidden tests;
- **fixable**: at least one reference fix passes all tests;
- **distinct fixes**: how many architecturally different reference fixes pass;
- **difficulty error**: how far the world's measured difficulty is from the target.

A world is **viable** only if it is broken, fixable, *and* at least two distinct fixes pass. Only viable worlds are served to contestants.

2.11 Evolution

Evolution here is a *genetic-algorithm-style* search over worlds. VANTA generates a small population (6 candidates) with slightly different genes and evaluates them all in parallel. It keeps the best half and creates new candidates by *mutating* the survivors' genes. It repeats for a few generations (3) and stops early as soon as it finds a viable world close to the target difficulty. This is how one generator serves both beginners and experts without any stored problem bank.

2.12 Probe

A **probe** (an experiment) is a small JSON description of a situation, written by the contestant. VANTA turns it into a custom C harness, compiles it with either the *original* system or the contestant's *edited* code, runs it, and returns a trace of what happened. Example:

```
{"days": 3, "jobs": [{"minutes": 10, "at": 0, "repeat": 50, "every": 5},
                     {"minutes": 300, "at": 30}]}
```

2.13 Witness

A **witness** is a person who "lived through" one real run of the broken system. They aren't a chatbot with a backstory. When a world's witnesses are first asked something, VANTA builds a busy period of realistic usage (a week of jobs, or a day of checkouts and returns). It runs the *actual starter code* on that period. Each witness then reports only what they could see from their own vantage point. One example is "I submitted one long render on day 1 at 08:30; it started on day 7". Asking the same question twice gives the same answer. Questions about the code are politely declined.

Replay means asking a witness to live through the *same* period again under the contestant's edited code, so the contestant can see how that person's experience changes.

2.14 Rating and calibrated difficulty

Every person has a **rating** (starting at 1100), updated like a chess rating. A world of difficulty *d* counts as an opponent rated `800 + 1200·d`. Solving a world above your level raises your rating a lot. A failed attempt on an easy world lowers it a little. The **recommended difficulty** for your next world is slightly above your current level.

2.15 Cohort

A **cohort** is a group session, such as a class or a hackathon room. A host opens it and gets a six-letter **join code**. Everyone who joins gets the *same defect in the same setting*, but in *their own world*, with different

names and numbers. The host console shows each person's progress and how many different approaches the accepted solutions took.

2.16 Originality

Originality analysis estimates how many genuinely different approaches were accepted in a cohort. It normalizes each solution against its own world and removes the parts copied from the starter. Then it compares what each person actually wrote. It measures *diversity*, not plagiarism.

2.17 Identity

VANTA has no accounts or passwords. The web UI gives each browser a random 8-character id stored in `localStorage` (`vanta_uid`). The Discord bot uses `discord:<user id>`. Ratings, worlds and cohort memberships are keyed by these ids.

CHAPTER 3

Architecture

3.1 The big picture

VANTA has three runnable parts:

1. **The backend** (`backend/`): a Python web service built with FastAPI. It generates worlds, compiles and runs C, answers witness questions, keeps ratings and cohorts, and stores everything in one SQLite file. It listens on port **8811**.
2. **The web frontend** (`frontend/`): a React + TypeScript single-page app built with Vite. In development it runs on port **5173** and forwards every `/api/...` request to the backend.
3. **The Discord bot** (`discord_bot/`): a separate Python process that turns `/vanta` slash commands into calls to the same HTTP API.

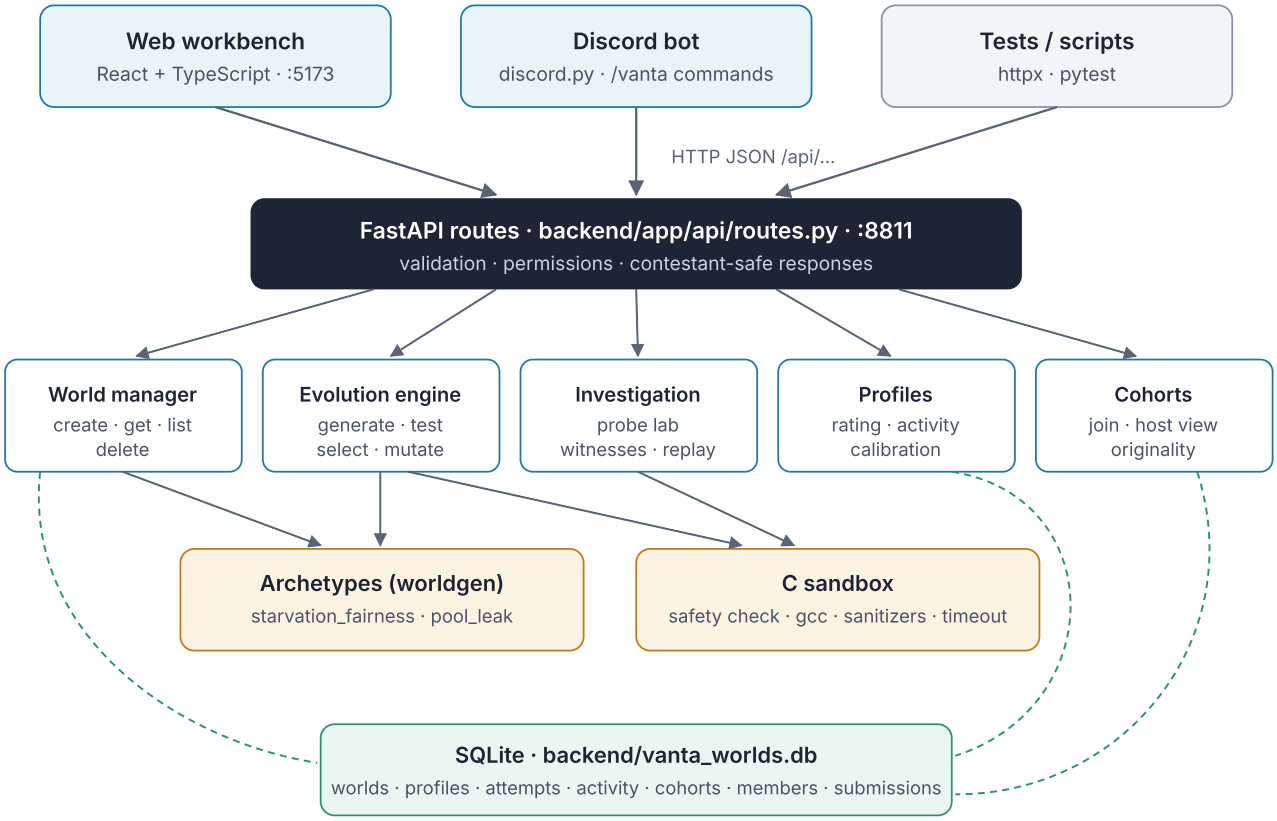


Figure 3.1. Components and how they call each other. Solid arrows are calls; dashed lines are database access.

3.2 Technology stack

Layer	Technology	Why
Web API	Python 3.13, FastAPI, Pydantic, Uvicorn	Async request handling; automatic request validation
C execution	`gcc` with AddressSanitizer + UndefinedBehaviorSanitizer	Real compilation and execution; memory errors become failures

Layer	Technology	Why
Storage	SQLite (Python's built-in `sqlite3`)	One file, no server to install
Frontend	React 19, TypeScript 6, Vite 8, Tailwind CSS 4, React Router 7	Fast development server; typed API client
Discord	discord.py 2.7, httpx	Slash commands; the bot is an HTTP client like the web UI
Testing	pytest, pytest-asyncio, httpx `ASGITransport`, Playwright (manual browser checks)	Tests run the real API in-process without a network

3.3 The contestant's journey

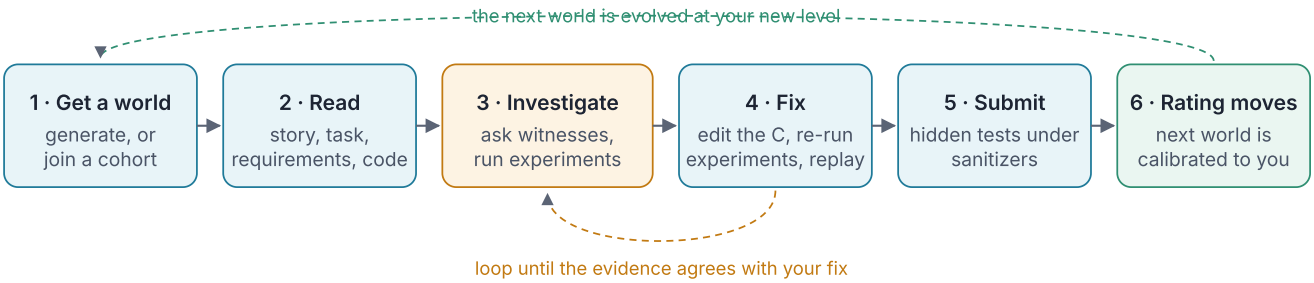


Figure 3.2. What a contestant does, and how the system adapts.

3.4 What happens on each request

Creating a world — `POST /api/worlds`

1. `routes.create_world` reads `created_by`, `target_difficulty` (or `null`), and optionally `archetype_id`.
2. If `target_difficulty` is `null`, `ProfileStore.recommended(created_by)` turns the person's rating into a difficulty. The response then says `"calibrated": true`.
3. `WorldManager.create_world` picks a random seed and calls `registry.generate_world`.
4. `generate_world` chooses the archetype (the one requested, or one drawn from the seed). It then calls `EvolutionEngine.evolve` with population 6 and 3 generations.
5. The engine generates candidates and measures each one's fitness with the *fast* sandbox. That means compiling the starter with the harness and compiling each reference fix with the harness. It returns the best viable world. If none is viable, `generate_world` retries with a new seed (up to 3 attempts).
6. The world is serialized to JSON and stored in the `worlds` table with its fitness report.
7. The contestant-safe view of the world is returned. It contains no ground truth, genes or params.

Submitting a fix — `POST /api/worlds/{id}/submit`

1. The stored world is loaded.
2. `evaluate_submission` sends the harness and the contestant's code to the *sanitized* sandbox.
3. The sandbox runs the static safety check, writes both files (each prefixed with the shared header) to a temporary folder, compiles, runs with a 15-second limit, and deletes the folder.
4. The `VANTA_TEST_RESULT` lines are parsed. The verdict is `accepted` only if `VANTA_OVERALL|1` was printed.
5. `ProfileStore.record_submission` updates the person's rating. `CohortStore.record_submission` stores the source if the world belongs to a cohort.
6. The response lists checks: public ones by name, hidden ones by number. It also includes the rating change and, for compile errors and timeouts, the compiler or runtime message.

Running an experiment — `POST /api/worlds/{id}/probe`

1. The archetype's `ProbeKit` validates the JSON scenario. Every value must be a whole number in a range. Repeats are expanded.
2. The kit writes a custom C harness with the scenario baked in as constant data.
3. The sandbox compiles it with the original starter (if `c_source` is `null`) or the contestant's code, and runs it.
4. The harness prints `PROBE_...` lines. The kit parses them into a trace (jobs, events, statistics, or operations and slots).
5. The activity is logged for the person's profile and the cohort host console.

Asking a witness — `POST /api/worlds/{id}/witnesses/{key}/ask`

1. The archetype's `WitnessKit` rebuilds the cast and the busy period deterministically from the world.
2. A free-text question is routed to a topic by keyword. Questions about the code or the fix get a polite deflection.
3. The busy period is run once with the starter code. The trace is cached per world and code hash. The answer for this witness and topic is computed from the trace.
4. If `c_source` is given, the same period is run again with that code and a second answer is returned under `replay`.

Joining a cohort — `POST /api/cohorts/{code}/join`

1. If the person is already a member and their world still exists, they get it back.
2. Otherwise a seed is derived from the cohort seed and the person's id. A world is evolved with the cohort's pinned variant and setting. If its identifiers clash with another member's, VANTA tries again with the next seed (up to 6 times).
3. The membership is recorded, and the world is returned.

3.5 Repository layout

```
vanta/
├── README.md                 project overview and quick start
├── run_dev.sh                starts API (:8811) and web UI (:5173) in the background
├── .gitignore
├── backend/
│   ├── main.py               FastAPI application object
│   ├── pytest.ini            test configuration
│   ├── app/
│   │   ├── api/routes.py     every HTTP endpoint
│   │   ├── core/             world store, profiles, cohorts, originality
│   │   ├── evolution/        evolution engine, reference fixes
│   │   ├── execution/        C sandbox
│   │   ├── investigation/    probe lab, witnesses
│   │   └── worldgen/         archetype base, registry, evaluator, archetypes/
│   └── tests/                35 automated tests
├── discord_bot/              bot.py, commands.py, vanta_api.py
├── frontend/
│   ├── index.html, vite.config.ts, package.json, tsconfig*.json
│   └── src/                  App, pages/, components/, api/, lib/, types.ts, index.css
├── docs/                     this documentation and its sources
└── scenarios/                (empty, reserved)
```

CHAPTER 4

Features in depth

This chapter explains every feature: what it does, how it works inside, and the exact numbers the code uses. Where a feature has a "why", it is given too, because the why is usually what a professor or judge will ask about.

4.1 World generation (archetypes)

The archetype contract

Every archetype subclasses `FlawArchetype` (`backend/app/worldgen/archetype.py`) and provides:

Member	Meaning
`id` , `title` , `domain`	Machine id (`pool_leak`), human title (*Resource Pool Integrity*), category
`variants`	List of bug variant names it can plant
`domain_count`	How many story settings it has
`generate(seed, genes=None, pins=None)`	Builds a complete `GeneratedWorld` ; must be deterministic
`rng(seed)`	A `random.Random` seeded with `"<id>:<seed>"`
`resolve_genes(genes)`	Fills in missing genes from `DEFAULT_GENES` and clamps every value to 0–1
`random_pins(rng)` , `apply_pins(...)`	Choose and apply a pinned variant and setting (used by cohorts)

The generated world has these fields. `GeneratedAgentGrounding` and `agent_groundings` are a leftover from the earlier chatbot design. They are generated but not stored or used.

Field	Contents
`archetype_id` , `seed` , `genes`	Where the world came from
`title` , `briefing` , `specification`	The story and the task
`starter_source`	The broken C the contestant starts from
`function_name` , `function_signature`	The function the contestant must implement
`shared_header`	Struct and prototypes, prepended to both C files
`mandatory_properties`	Requirements (plain English)
`optional_objectives`	Nice-to-have goals (not graded)
`sample_input_description`	One line about scale, such as "A day is 436 simulated minutes"
`harness_source`	The hidden test program
`ground_truth_summary`	Admin-only one-line summary of the planted bug and constants
`params`	Structured names and constants used by the probe lab, witnesses and cohorts

Archetype 1: Shared Resource Fairness (`starvation_fairness`)

Story. One machine is shared by everyone, opens for part of each day, and runs one job at a time. People complain that "certain people dominate it". The settings are:

#	Job noun	Machine	Place
0	print job	3D printer	the student fabrication lab
1	wash cycle	washing machine	the dormitory laundry room
2	render job	shared GPU workstation	the media lab
3	repair request	bike repair stand	the campus cycling co-op
4	scan request	document scanner	the library digitization desk

Names. The struct name comes from 12 options (`Job` , `Task` , `Request` , `WorkItem` , `Ticket` , `Order` , `Booking` , `Submission` , `QueueEntry` , `Batch` , `Run` , `Commission`). The function name is a verb (8 options: `choose` , `select` , `pick` , `schedule` , `dispatch` , `elect` , `decide` , `nominate`) plus a tail (7 options: `Next` , `NextJob` , `ToRun` , `Upcoming` , `Candidate` , `NextUp` , `Successor`). That gives 672 combinations, for example `electUpcoming` .

The struct (in the shared header) always has these fields:

```
struct Job {
    int id;
    int minutesNeeded;   /* how long the job takes */
    int submittedAt;     /* simulated minute it was handed in */
    int isAbandoned;     /* owner walked away */
    int isFinished;
    int isRejected;      /* longer than a whole day: refused up front */
    int startedAt;       /* -1 until started */
    int finishedAt;      /* -1 until finished */
};
int chooseToRun(struct Job jobs[], int count, int now, int minutesLeftToday);
```

The contestant's function receives every job, the current minute and the minutes left before closing. It returns the index of the job to start, or -1 for none.

Constants. Every world draws its own numbers:

Constant	How it is drawn
Day length *L*	One of 360/420/480/540/600, multiplied by `0.8 + 0.6 × constant_magnitude`
Long job	Random between `0.55·L` and `L − 10` minutes
Short job	5–15 minutes
Short-job interval	2 to (short job − 1) minutes, so the queue never drains
Long job submitted at	interval × (8–20), into a stream that is already flowing
Starvation bound	1500–2500 minutes of waiting counts as "ignored forever"

Variants. `bug_subtlety` biases the choice. The weights are `[1 − s, 0.5, 0.3 + s]` for `no_close_check` , `pure_shortest_first` and `truncating_priority` .

```
/* pure_shortest_first: always the shortest eligible job */
if (best == -1 || jobs[i].minutesNeeded < jobs[best].minutesNeeded) best = i;

/* no_close_check: oldest first, but never looks at minutesLeftToday */
if (oldest == -1 || jobs[i].submittedAt < jobs[oldest].submittedAt) oldest = i;
```

```
/* truncating_priority: integer division collapses the score to 0 */
int score = (waited > 0) ? (jobs[i].minutesNeeded / waited) : jobs[i].minutesNeeded;
```

Requirements shown to the contestant: no valid job is ignored forever; an abandoned job frees the machine; never start a job that can't finish before close; reject a job longer than a full day; always terminate; never start a job twice.

Hidden tests (8). The simulation runs up to 120 days, one tick per minute. A violation bitmask records 1 = starvation, 4 = started too late and 16 = a bad pick (out of range, not yet submitted, or already done or started). Every test fails on any of these.

Test	Scenario
`public_simple`	Three short jobs a few minutes apart
`public_single`	One 30-minute job
`hidden_zero_jobs`	Empty list
`hidden_exact_fit`	One job exactly one day long
`hidden_oversized_rejected`	A job longer than a day must be rejected; a normal one must finish
`hidden_starvation_stream`	Up to 600 short jobs arriving faster than they finish, plus one long job submitted mid-stream
`hidden_abandonment`	An abandoned job must not block the next one
`hidden_reverse_order`	Jobs given in reverse submission order

Archetype 2: Resource Pool Integrity (`pool_leak`)

Story. A fixed number of units is checked out and returned all day. The system claims to be full when plenty should be free. The settings are:

#	Item	Unit	Place
0	vehicle	parking spot	the campus parking garage
1	book	checkout slot	the university library
2	kit	locker	the equipment rental room
3	laptop	loaner unit	the IT help desk

Names. The struct name comes from 12 options. The checkout function is one of 8 verbs (`checkout` , `acquire` , `reserve` , `take` , `claim` , `lease` , `borrow` , `grab`) plus one of 6 nouns. The release function is one of 8 verbs (`release` , `return` , `free` , `giveBack` , `surrender` , `drop` , `vacate` , `handBack`) plus one of 6 nouns. The struct is always `{ int inUse; int holderId; }` .

Constants. The pool size is one of 10/12/16/20/24, multiplied by `0.7 + 0.8 × constant_magnitude` , with a minimum of 6. The churn test runs `200 + 400 × edge_density` cycles.

Variants (chosen uniformly):

```
/* linear_search_first_match: frees a slot whose OLD holderId matches,
   even if the slot is already free -> a second release "succeeds" */
if (pool[i].holderId == holderId) { pool[i].inUse = 0; return 1; }

/* off_by_one_index: the wrong modulus */
```

```
int idx = holderId % (poolSize + 1);

/* no_double_checkout_guard: checkout never asks "do you already hold one?" */
```

Requirements: a released unit becomes available again; two checkouts never hold the same unit; checkout fails cleanly when the pool is truly full; the held count equals checkouts minus matched releases; always terminate; releasing a never-checked-out unit is rejected; a second release of the same unit is rejected.

Hidden tests (8): `public_basic`, `public_full_rejected`, `hidden_churn_no_leak`, `hidden_reverse_release`, `hidden_release_never_checked_out`, `hidden_double_release`, `hidden_no_aliasing` (the same holder can't check out twice) and `hidden_capacity_reusable` (fill, empty, refill).

The registry

`backend/app/worldgen/registry.py` holds `ARCHETYPE_POOL` (the list of archetype objects) and `ARCHETYPES_BY_ID`. `generate_world(session_seed, target_difficulty, archetype_id, engine, max_restarts=3, pins=None)` picks the archetype. It uses the requested one, or draws one with `random.Random("archetype_pick:<seed>")` so the defect *class* itself varies. It then runs evolution. If no viable world is found it retries with seed `seed × 7919 + attempt`.

4.2 The evolution engine

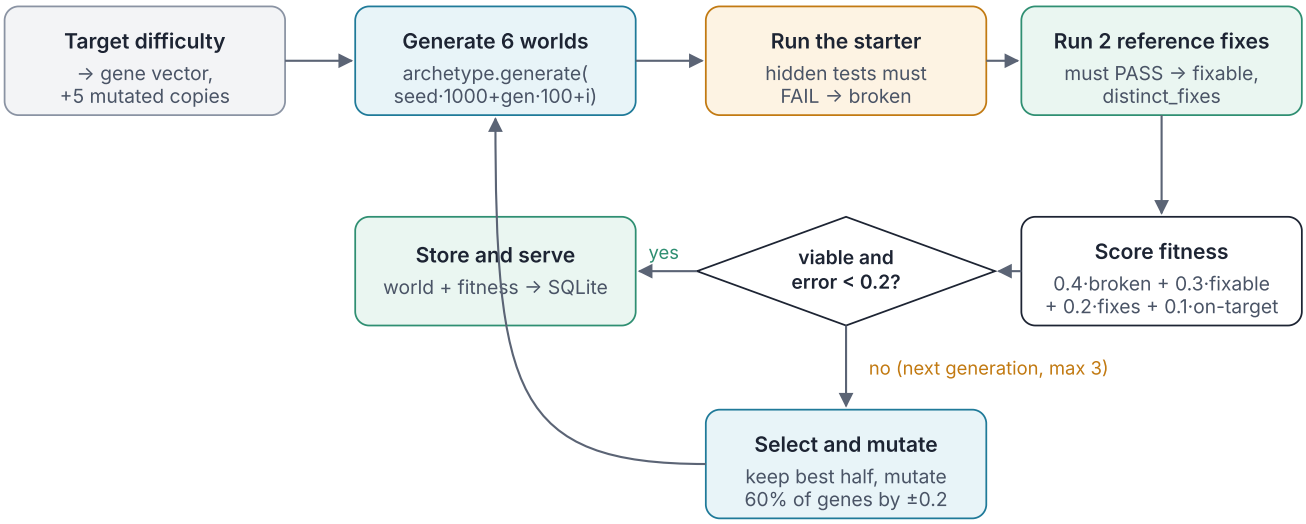


Figure 4.1. One run of `EvolutionEngine.evolve`. Candidates in a generation are evaluated in parallel.

Step by step

1. **Gene vector.** `_target_vector(t)` maps a difficulty *t* to genes: `system_size = t`, `bug_subtlety = t`, `constant_magnitude = 0.3 + 0.6t`, `noise = t`, `edge_density = 0.3 + 0.6t` and `reasoning_depth = t`.
2. **Initial population.** The base vector plus 5 mutated copies. `_mutate` changes each gene with probability 0.6 by a random amount in ±0.25, then clamps it to 0–1.
3. **Generate.** Candidate *i* in generation *g* uses seed `base_seed × 1000 + g × 100 + i`.
4. **Measure fitness** (`_measure_fitness`, run for all candidates at once with `asyncio.gather`): - run the starter against the harness. A compile error means the score is 0. **Broken** means the verdict is "rejected" and at least one test failed; - run every reference fix. **Fixable** means at least 1 passes, and **distinct_fixes** counts how many pass; - **difficulty error** = |estimated − target|. The estimate is `0.6 × gene_part + 0.4 × observed_part`, where `gene_part = 0.5·bug_subtlety + 0.2·constant_magnitude + 0.3·edge_density`.

`observed_part = 1 − (fraction of starter tests failing)`, because a starter that fails everything is an *obvious* bug.

5. **Score** = `0.4·broken + 0.3·fixable + 0.2·min(distinct, 2)/2 + 0.1·(1 − error)`.
6. **Early exit.** If any candidate is viable and its difficulty error is below 0.2, return it (preferring more distinct fixes, then lower error).
7. **Select and mutate.** Keep the top half (at least 2). Fill the population back up by mutating random survivors with strength 0.2. Repeat.
8. If no generation produced a near-target viable world, return the best candidate seen. The registry then retries with a new seed if that candidate isn't viable.

Speed. The loop uses the fast (unsanitized) sandbox with a 20-second limit, because it compiles 3 programs per candidate (starter + 2 fixes) × 6 candidates × up to 3 generations. In practice a pool world takes about 1 second and a scheduler world about 5–30 seconds.

Why this matters. Nobody writes problems by hand, and there is no answer key to leak. The valuable part is the generate–test–select loop, the fitness function and the archetypes, not any one world.

4.3 Grading submissions

The sandbox

`CSandbox(timeout_seconds, compiler="gcc", fast=False).compile_and_run(harness, candidate, shared_header)` does the following:

1. Runs `static_safety_check(candidate)`. Any match of a forbidden pattern returns a compile error without compiling. The forbidden patterns are `system(`, `exec*(`, `fork(`, `popen(`, `fopen(`, `freopen(`, `remove(`, `unlink(`, `rename(`, `#include <unistd.h>`, `#include <sys/...>`, `#include <fcntl.h>`, `asm` and `__asm__`.
2. Creates a temporary folder and writes `harness_main.c` and `candidate.c`, each prefixed with the shared header. Each `.c` file is compiled separately, so both need the struct definition.
3. Compiles both into one binary. Sanitized mode uses `-Wall -Wextra -fsanitize=undefined,address -O0 -g`; fast mode uses `-w -O1`.
4. Runs the binary with a wall-clock timeout. A timeout kills it.
5. Returns a `CRunResult` with `success`, `stdout`, `stderr`, `compile_error`, `runtime_error`, `timed_out` and `exit_code`. The folder is always deleted.

Turning output into a verdict

`evaluate_submission(world, source, sandbox)` (`backend/app/worldgen/evaluate.py`):

Situation	Verdict	Reason
Compilation failed (or the safety check refused it)	rejected	`compile_error` (message returned)
Ran out of time	rejected	`timeout`
No `VANTA_OVERALL|` line printed (the program crashed)	rejected	`runtime_error`
`VANTA_OVERALL|1`	accepted	—
`VANTA_OVERALL|0`	rejected	`failed_tests`

The harness deliberately exits with code 1 when a test fails. A non-zero exit is therefore normal for a wrong answer, and VANTA always reads the printed results first.

What the contestant sees

`routes._contestant_tests` renames checks before they leave the server. `public_basic` becomes **"public: basic"**, and every hidden test becomes **"hidden check N"**. The web UI shows checks as chips, separately from the requirements list, because one hidden check can test several requirements at once.

4.4 Probe lab (experiments)

A **ProbeKit** exists per archetype (`backend/app/investigation/probes.py`). It has four jobs:

Method	Job
`describe(world)`	Returns help text, limits, and an example scenario (`GET /probe`)
`normalize(world, scenario)`	Validates the JSON and expands `repeat` entries; raises `ProbeError` with a contestant-safe message
`harness(world, norm, event_cap)`	Writes a C `main()` with the scenario baked in as constant data
`parse(world, norm, stdout)`	Turns `PROBE_...` lines into a trace dictionary

`run()` puts these together. A compile error, timeout or crash comes back as `ok: false` with `error` and `detail` . Sanitizer output is trimmed to 12 lines and sandbox paths are removed.

Scheduler experiments (`job_queue`)

```
{
  "days": 3,
  "jobs": [
    {"minutes": 20, "at": 0},
    {"minutes": 45, "at": 10},
    {"minutes": 10, "at": 15, "repeat": 4, "every": 30},
    {"minutes": 60, "at": 100, "abandon_after": 25}
  ]
}
```

Field	Rule
`days`	1–30 (default 3): how long to watch
`minutes`	1–20000
`at`	0 to `days × L` : the minute it is submitted (day 2 opens at minute *L*)
`repeat` , `every`	Submit a stream: `repeat` copies, `every` minutes apart (at most 700 jobs in total)
`abandon_after`	The owner walks away this many minutes after submitting

The generated harness runs the same simulation as the hidden tests, and also records events and statistics. The trace contains:

- **jobs**: for every job, its submitted, started and finished times, its wait, and its status (`finished` , `rejected` , `abandoned` , `running` or `never_started`);
- **events**: `start` , `finish` , `freed` , `abandoned` , `rejected` , `refused_too_late` , `invalid_pick` and `invalid_index` . Repeated refusals are logged once per job per day. Probe runs keep up to 1500 events;
- **stats**: busy minutes, open minutes, idle minutes while something was waiting, too-late attempts, invalid picks, days simulated, and the longest queue.

The web UI draws a day-by-day timeline. Each day is a row, each run is a coloured bar, and amber triangles mark refusals. It also shows a table summarizing each entry of the experiment.

Pool experiments (`pool`)

```
{"ops": [
  {"op": "checkout", "holder": 1},
  {"op": "release",  "holder": 1},
  {"op": "release",  "holder": 1},
  {"op": "cycle",    "holder": 100, "repeat": 40, "step": 7}
]}
```

`checkout` and `release` call the system once for a holder id (0–10,000,000). `cycle` does both. `repeat` + `step` expand one entry into many holders (holder, holder+step, ...). There are at most 3000 operations. After every operation the harness prints what the system returned, how many slots are marked in use, and the **receipts** count (checkouts granted minus releases accepted). It ends with the full slot table. The UI plots *in use* against *receipts*, highlights rows where they differ, and draws the final pool as a grid.

Safety

Scenarios never become C *text* written by the contestant. Every value must be a whole number in range, otherwise `ProbeError` is raised. The harness is assembled from those integers only, so a scenario like `"minutes": "5); system(\"x\"); ("` is refused before anything is compiled. This is tested in `test_probe_rejects_bad_scenarios`.

4.5 Witnesses

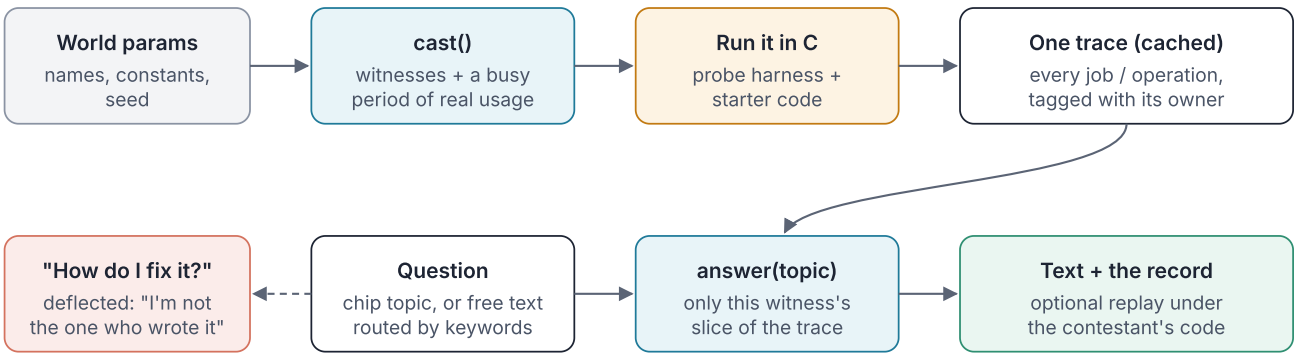


Figure 4.2. How a witness answer is produced. Nothing in the answer is written by hand; it is read from one execution.

Why witnesses exist

The old design had LLM characters with written backstories, and Discord sessions became arguments: the contestant trying to get a bot to concede a stored "fact". Witnesses remove the thing to argue about. They hold no facts, only a vantage point on one real run. Answers are deterministic, bounded, and backed by an evidence table ("show the record behind this").

The busy period for the scheduler

`StarvationWitnessKit.cast(world)` builds a 7-day load from the world's own constants, using a generator seeded with the world:

Owner	Load
`regular`	Short jobs every (short − 1) minutes during the first 50–65% of each day, for 6 days (at most 90 per day)
`big`	One long job submitted 15–60 minutes into day 1
`midday`	A medium job (10–18% of a day) around midday on days 1, 3 and 5
`late`	A job of 20–30% of a day submitted 8–15% of a day before closing, on days 1 and 4
`operator`	No jobs; reads the machine's log

Witness names are drawn from a pool of 24 first names, for example Priya, Arjun, Meera, Kabir and Ananya.

Witness	Can see	Questions (topics)
regular, big, midday, late	Their own jobs only	*story* (what happened), *worst* (worst wait), *timing* (when in the day jobs started), *machine* (what the machine ran while they waited, and whose jobs)
operator	The machine log	*utilization*, *refusals* (too-late or invalid picks), *queue* (longest queue, who is still waiting), *closing* (what happens near closing time)

Times are shown as clock times. Each day opens at 08:00, so minute 0 is day 1, 08:00.

The busy day for the pool

`PoolWitnessKit.cast(world)` builds one day of operations:

Owner	Operations
background	Pool size − 4 long-term holders check out in the morning
`regular`	6–9 borrow-and-return cycles
`staff`	Processes returns for about half the background holders, scanning every other one twice by accident, plus one scan of an id that never borrowed
`double`	Two people whose app froze, so each borrow request went through twice; they return once
`newcomer`	Four people trying to borrow at the end of the day

The actor timelines are merged randomly, but each actor's own order is preserved. The pool witnesses also reconstruct a **physical reality** from what people *did*, not what the system recorded. A person physically gets an item when their checkout is granted and they have nothing in hand. They physically hand one back whenever they go to return. This gives three numbers that should always agree:

1. **receipts**: checkouts granted minus returns accepted;
2. **in use**: what the pool memory says;
3. **physically out**: what people actually have.

Witness	Can see	Topics
regular	What the system told them	*story*, *denied* (told nothing available, while N were free), *returns* (were returns accepted)
staff	Each scan's response	*story*, *double* (what a second scan does), *unknown* (scanning someone who never borrowed)

Witness	Can see	Topics
double	What they received and what the app showed	*story* (duplicate requests), *returns*
newcomer	The shelf	*story*, *shelf* (how many were physically free)
auditor	Receipts, system and physical counts	*reconcile* (do the three agree), *first* (first transaction where they disagree), *stock* (end-of-day slot table: ghost slots, holders on two slots)

Routing and deflection

Clicking a question chip sends its `topic` directly. Free text is matched against each topic's keyword list, and the topic with the most hits wins. Text matching the deflection pattern is politely refused. The pattern covers words like *fix*, *solution*, *code*, *bug*, *algorithm*, *implement*, *should I/you/we*, *how do/to/would*, *what's wrong*, *root cause*, *why does*, *why is* and *patch*. Anything unmatched gets "I'm not sure what you're asking", plus the list of questions the witness can answer.

Replay and caching

When `c_source` is supplied and differs from the starter, the same busy period is run a second time with the contestant's code. The same topic is then answered again. The UI shows both answers side by side ("the original system" / "under your code"). A crash or compile error in the replay is reported as text, never as a server error.

Traces are cached in memory (up to 96 entries, least recently used evicted), keyed by world id and a SHA-256 hash of the code. Witness runs allow up to 8000 events. Caching a *result of running code* is not a fact store: the result can always be recomputed.

Trade-offs become visible. In a real test world, a fair fix cut the long-job wait from about 6 days to 10 minutes. The same fix raised the short-job user's typical wait from 24 minutes to over 5 hours, and that witness says so. Both fixes are correct, and they make different people happy.

4.6 Adaptive difficulty (ratings)

`backend/app/core/profiles.py` implements an Elo-style rating.

Rule	Value
Starting rating	1100
A world's rating	`800 + 1200 × difficulty` (difficulty 0.5 → 1400)
Expected score	`1 / (1 + 10^((world − you) / 400))`
Accepted submission	rating += K × (1 − expected), with K = 48 for your first 5 worlds, then 32
Rejected submission	rating += K × (0 − expected), with K = 8 for your first 5 worlds, then 5; only your first 5 rejections on a world count
Already solved this world	No change (no farming)
Recommended next difficulty	`(rating − 800) / 1200 + 0.06`, clamped to 0.10–0.95

A rejection can never *raise* a rating. Otherwise resubmitting the starter, which already passes some tests, would earn points. Levels: under 1000 *Novice*, under 1200 *Apprentice*, under 1400 *Practitioner*, under 1600

Engineer, under 1800 *Expert*, then *Master*.

The profile also counts **activity**: experiments run (`probe`), and witness questions (`witness`) and replays (`replay`). These are shown to the person and to cohort hosts.

Example. A new user (1100) solves a 0.45 world (rating 1340). Their expected score is about 0.20, so they gain 48 × 0.80 ≈ +38, reaching 1138. Their next calibrated world is 0.34.

4.7 Cohorts

`backend/app/core/cohorts.py` (`CohortStore`).

1. **Create.** The host gives a name, optionally a defect class, and a difficulty (default 0.45). VANTA draws a cohort seed and picks the archetype (if not given). It pins one variant and one setting, and allocates a unique **6-character join code**. The code alphabet leaves out look-alike characters (no I, L, O, 0 or 1).
2. **Join.** A member's seed is the first 4 bytes of `SHA-256("<cohort seed>:<user id>:<attempt>")` . A world is evolved with the cohort's pins and difficulty. If its identifier set (struct name + function names) matches another member's, it is deleted and the next attempt is tried, up to 6 times. So no two members share names, and copied code won't compile.
3. **Rejoin.** A member who joins again gets their same world back. If their world was deleted, they get a new one.
4. **Close.** No new members, but existing members can continue.
5. **Delete.** Removes the cohort, its members, its submissions, and every member's world.
6. **Host view.** For each member: identifiers, number of submissions, best checks passed, solved or not, and experiment and witness counts. For the whole cohort: the setting, the planted variant (hidden behind a "reveal" button in the UI), and the originality report.

Only the host (matching `host_id`) or an admin can close, delete, or see the host view.

4.8 Originality analysis

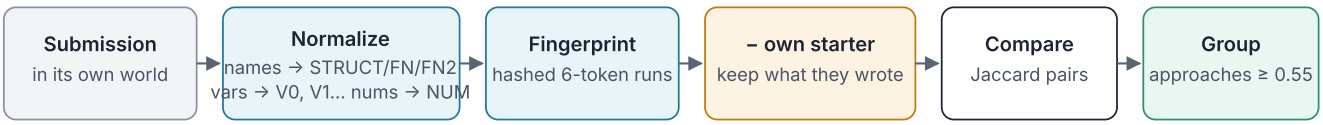


Figure 4.3. The originality pipeline in `backend/app/core/originality.py` .

The problem is that every cohort member's code uses different names and different numbers, but everyone also starts from the same shaped starter. Comparing raw text would call everyone different. Comparing naively would call everyone the same. The steps, in the spirit of the well-known MOSS plagiarism detector, are:

1. **Strip** comments, string and character literals, and `#` lines.
2. **Tokenize** the C into identifiers, numbers and operators.
3. **Normalize:** - the world's own struct name becomes `STRUCT` , the main function `FN` , and the checkout function `FN2` ; - C keywords and the struct's field names are kept; - every other identifier becomes `V0` , `V1` , … in order of first appearance, so renaming variables changes nothing; - numbers other than 0 and 1 become `NUM` .
4. **Fingerprint:** hash every run of 6 consecutive tokens (BLAKE2b, 8 bytes).

5. **Subtract the starter:** remove the fingerprints of the member's own normalized starter. What remains is what that person actually added or changed. If fewer than 3 remain, the submission counts as "unchanged from starter".
6. **Compare** every pair with Jaccard similarity (shared ÷ total).
7. **Group** accepted solutions by single linkage: a solution joins a group if it is at least 0.55 similar to any member. The number of groups is the number of **distinct approaches**. Pairs at 0.85 or above are listed as **near-identical**, for the host to check.

The tests check this on real worlds. The same forward-scan fix written in two different worlds with renamed variables scores above 0.85. A forward-scan fix against a backward-scan fix scores below 0.55.

4.9 The web workbench

Page (route)	What it does
Landing (`/`)	Choose a defect class (or *Surprise me*) and a difficulty (*Calibrated to you*, Gentle, Standard, Hard, Brutal), then generate. Join a cohort by code, host a cohort, see cohorts you host, and see your open worlds (open or delete them)
World (`/worlds/:id`)	Situation, task, signature, requirements, a cohort chip if relevant, an **Investigate** panel with *Witnesses* and *Probe lab* tabs, the source editor (with *Reset to original*), *Submit fix*, the verdict, checks and rating change
Cohort join (`/cohorts/:code`)	Cohort name, defect class and difficulty. Enter a display name to join, or open your world if already a member
Host console (`/cohorts/:code/host`)	Large join code, join link and Discord command; counts (joined, submitted, solved, distinct approaches); approach groups; most similar pairs; the member table; reveal the planted defect; close or delete

The header shows your level and rating (`ProfileBadge`). The badge refreshes when a submission fires the `vanta:profile-changed` browser event.

The visual language is a "blueprint / forensic workbench". It uses a deep ink-blue page with a faint grid and cool neutrals for structure. One warm amber colour is reserved for "something is wrong, look here". Teal means accepted and rust means rejected.

4.10 The Discord bot

All commands live under `/vanta` . Slow commands are *deferred* (Discord shows "thinking..."), because generating a world can take longer than Discord's 3-second reply limit.

Command	Options	Notes
`create`	`defect` (Surprise me / Shared Resource Fairness / Resource Pool Integrity), `difficulty` (Calibrated to me / Gentle / Standard / Hard / Brutal)	Replies with an embed and the starter as `<function>.c`
`list`	—	Your worlds (private reply)
`info`	`world_id`	Briefing, requirements, signature, starter file
`submit`	`world_id` , `file` (.c attachment, up to 64 KB) or `code` (```c fences accepted)	Verdict, checks, rating change

Command	Options	Notes
`delete`	`world_id`	Only your own, unless you have Manage Server
`delete_all`	—	All of *your* worlds
`admin_delete_all`	—	Manage Server only; every world
`witnesses`	`world_id`	Who was there and what to ask (private)
`ask`	`world_id`, `witness` (autocomplete), `question`	One answer plus up to 8 rows of the record
`probe`	`world_id`, `scenario` (JSON, omit for help), `file`	Runs on the original, or on your attached code
`profile`	—	Level, rating, counts, next difficulty
`cohort_create`	`name`, `defect`, `difficulty`	Posts the join code
`join`	`code`	Your world arrives privately; uses your Discord display name
`cohort_status`	`code`	Host only: members, progress, approaches, near-identical pairs
`cohort_close` / `cohort_delete`	`code`	Host only

Discord users are identified as `discord:<numeric id>`. These commands manage whole VANTA worlds. They never create or delete Discord channels.

4.11 Permissions and admin

Action	Who may do it
Read a world, probe it, ask its witnesses, submit to it	Anyone with the world id
Delete a world	Its creator (`requested_by` must match `created_by`) or an admin
Delete all of one person's worlds	That person or an admin
Delete every world	Admin only
See ground truth (`/api/admin/worlds/{id}`)	Admin only
Close, delete or view the host console of a cohort	Its host or an admin

A request is **admin** when it carries the `X-Vanta-Admin` header. If the server has `VANTA_ADMIN_TOKEN` set, the header must equal that token, compared in constant time. If no token is configured (local demo mode), any `X-Vanta-Admin` header is accepted. The Discord bot sends the header only for users with the *Manage Server* or *Administrator* permission.

CHAPTER 5

File-by-file reference

This chapter goes through **every file in the repository**, grouped by folder. For each one you get its size, its purpose, what it contains, and which other files depend on it. Line counts are read from the repository when this document is built.

Files that are **generated** or **downloaded** (and not tracked by git) are covered at the end: the database, logs, `node_modules/`, `.venv/`, `dist/` and caches.

5.1 Repository root

`README.md`

142 lines · Markdown

The front page of the project. It covers the motto, the four layers (archetypes, evolution, investigation, adaptation), why witnesses can't be argued with, and why two people's solutions differ. It also covers cohorts, the Discord command table, admin rights, how to run everything, an API summary table, and a five-minute demo. Read it first. This document expands on it.

`run_dev.sh`

9 lines · Bash

Starts the API (`uvicorn main:app` on 127.0.0.1:8811) and the web dev server (`npm run dev` on 127.0.0.1:5173), both in the background with `setsid nohup`. Output goes to `backend/server.log` and `frontend/vite.log`. Requires that `backend/.venv` and `frontend/node_modules` already exist.

`.gitignore`

13 lines

Keeps generated and local files out of git: `__pycache__/`, `*.pyc`, `.venv/`, `node_modules/`, `dist/`, `*.db` (the SQLite database), `.env`, `build/`, `*.o`, `*.out`, `.pytest_cache/` and `*.log`.

5.2 backend/ — the API service

`backend/main.py`

20 lines · Python

Creates the FastAPI application (`title="VANTA"`, `version="0.4.0"`). It adds CORS middleware that allows any origin, so the web UI and tools can call it, and mounts the router from `app/api/routes.py`. It also defines `GET /health`, which returns `{"status": "ok"}`. Uvicorn loads `main:app`.

`backend/pytest.ini`

2 lines

Sets `asyncio_mode = auto`, so `async def test_...` functions run without decorators.

`backend/app/__init__.py` and every other `__init__.py`

0 lines each

Empty files that make each folder a Python *package*, so `from app.core.profiles import ...` works. They exist in `app/`, `app/api/`, `app/core/`, `app/evolution/`, `app/execution/`, `app/investigation/`, `app/worldgen/`, `app/worldgen/archetypes/` and `tests/`.

5.3 backend/app/api/

`backend/app/api/routes.py`

348 lines · Python · FastAPI

Every HTTP endpoint lives here (see Chapter 6 for the full reference). It creates the long-lived objects shared by all requests:

Object	What it is
`router`	`APIRouter(prefix="/api")`
`_manager`	`WorldManager(engine=EvolutionEngine())`, which uses the fast sandbox for generation
`_eval_sandbox`	`CSandbox(timeout_seconds=15.0, fast=False)`, the sanitized sandbox for contestants
`_profiles`	`ProfileStore()`
`_cohorts`	`CohortStore(_manager)`

Request bodies are Pydantic models: `CreateWorldRequest`, `SubmitRequest`, `ProbeRequest`, `AskRequest`, `CreateCohortRequest`, `JoinCohortRequest` and `HostRequest`. The helpers are:

Helper	Purpose
`_is_admin(token)`	Checks the `X-Vanta-Admin` header against `VANTA_ADMIN_TOKEN` (any header in demo mode)
`_cohort_call(fn, ...)`	Turns `CohortError` subclasses into HTTP 400/403/404
`_record(world_id)`	Loads a world or raises 404
`_summary(r)`, `_public(r)`	The contestant-safe views of a world. They never include ground truth, genes or params
`_cohort_link(world_id)`	Adds `{code, name, display_name}` if the world belongs to a cohort
`_contestant_tests(tests)`	Renames checks: "public: ..." or "hidden check N"

The endpoints are the archetype list, world create/list/get/submit/delete, probe info and run, witness list and ask, profiles, cohorts (create, list, get, join, host, close, delete), and the admin world view.

5.4 backend/app/core/ — state and bookkeeping

`backend/app/core/world_manager.py`

192 lines · Python · SQLite

Creates, stores, reads, lists and deletes worlds.

Name	Purpose
`DB_PATH`	`backend/vanta_worlds.db`. Every store reads it at call time, so tests can redirect it

Name	Purpose
`_connect()`	Opens SQLite and creates the `worlds` table if needed
`_WORLD_FIELDS`, `_serialize_world`, `_deserialize_world`	Convert a `GeneratedWorld` to and from JSON. `params` is included; `agent_groundings` is not stored
`WorldRecord`	A stored world: id, seed, archetype, title, creator, target difficulty, viable flag, created time, world and fitness
`WorldNotFoundError`	Raised for unknown ids
`WorldManager.create_world(created_by, seed, target_difficulty, archetype_id, pins)`	Evolves a world through the registry and stores it with its fitness report
`get_world`, `list_worlds(created_by)`, `delete_world`, `delete_all_worlds(created_by)`	Read and remove worlds

`backend/app/core/profiles.py`

191 lines · Python · SQLite

Ratings and activity for adaptive difficulty (see section 4.6).

Name	Purpose
`START_RATING`, `BASE`, `SPAN`, `_LEVELS`	1100; 800; 1200; the level names and thresholds
`world_rating(d)`, `recommended_difficulty(r)`, `level_for(r)`	The rating maths
`_connect()`	Creates the `profiles`, `attempts` and `activity` tables
`RatingChange`	`before`, `after`, `counted`
`ProfileStore.rating / recommended`	Read a person's rating, or the difficulty to aim at
`ProfileStore.log_activity(user, world, kind)`	Records `probe`, `witness` or `replay`
`ProfileStore.record_submission(...)`	Applies the Elo update with the anti-farming rules and stores the attempt
`ProfileStore.profile(user)`	Everything the profile endpoint returns, including the last 12 attempts

`backend/app/core/cohorts.py`

385 lines · Python · SQLite

Group sessions (see section 4.7).

Name	Purpose
`CohortError`, `CohortNotFound`, `CohortForbidden`	Errors carrying an HTTP status (400, 404, 403)
`Cohort`	Dataclass: id, code, name, host, archetype, pins, difficulty, seed, status and created time; `public()` gives the safe view
`_connect()`	Creates `cohorts`, `cohort_members`, `cohort_submissions` and `activity`
`_identifiers(world)`, `_signature(world)`	A member's struct and function names, used to guarantee they are unique

Name	Purpose
`CohortStore.create(host, name, archetype, difficulty)`	Draws the seed and pins and allocates a join code
`get`, `list_for_host`, `member_count`, `membership`	Lookups. Codes are case-insensitive
`join(code, user, display_name)`	Rejoin, or evolve a unique world (up to 6 attempts to avoid name clashes)
`close`, `delete`	Host or admin only. Delete also removes member worlds
`for_world(world_id)`	Which cohort and member a world belongs to
`record_submission(...)`	Stores the verdict and source for cohort worlds
`host_view(code, requester, is_admin)`	Members with progress and activity, setting, planted variant, and originality report

`backend/app/core/originality.py`

155 lines · Python

Approach-diversity analysis (see section 4.8). The constants are `K = 6` (tokens per fingerprint), `APPROACH_THRESHOLD = 0.55` and `NEAR_IDENTICAL = 0.85`. The functions are: `_strip` (remove comments, strings and preprocessor lines); `_world_names` and `_fields` (what to canonicalize and what to keep); `normalize`; `fingerprints`; `novel_fingerprints` (submission minus starter); `jaccard`; and `analyze(entries)`. `analyze` returns `submissions_compared`, `unchanged_from_starter`, `accepted`, `distinct_approaches`, `approaches`, `pairs` (top 25), `near_identical` and a `method` description.

5.5 backend/app/evolution/ — the generator's search loop

`backend/app/evolution/engine.py`

214 lines · Python · asyncio

Name	Purpose
`FitnessReport`	`broken`, `fixable`, `distinct_fixes`, `difficulty_error`, `score`, `notes`, and the `viable` property
`EvolvedWorld`	A world, its fitness, and the generation it came from
`EvolutionEngine(sandbox=None)`	Defaults to `CSandbox(timeout_seconds=20.0, fast=True)`
`_target_vector(t)`	Difficulty → genes
`_mutate(genes, rng, strength)`	Changes about 60% of genes by up to ±strength
`_measure_fitness(archetype, world, target)`	Runs the starter and the reference fixes; computes the score
`_estimate_difficulty(world, starter_eval)`	Blends gene difficulty with how many starter tests fail
`evolve(archetype, base_seed, target_difficulty, population, generations, pins)`	The loop in Figure 4.1

`backend/app/evolution/reference_fixes.py`

122 lines · Python (generates C)

Known-good solutions, used **only** inside the evolution loop to prove fixability. Each function reads the world's randomized names (from the shared header) and returns C source:

Function	Approach
`_starvation_aging_guard`	Shortest job first, but any job that has waited 1200+ minutes is forced through; respects closing time
`_starvation_oldest_first`	Plain first-come-first-served among jobs that fit before closing
`_pool_forward`	Correct checkout (refuses a holder who already has a slot) and release (must be in use *and* match), scanning 0 → n−1
`_pool_backward`	The same rules scanning n−1 → 0, which gives different slot assignments under load

`REFERENCE_FIXES` maps an archetype id to its list of fix functions.

5.6 backend/app/execution/

`backend/app/execution/c_sandbox.py`

157 lines · Python · gcc

Compiles and runs C (see section 4.3). It contains `FORBIDDEN_PATTERNS` (14 regular expressions), `UnsafeCSourceError`, `static_safety_check(source)`, `CRunResult`, and `CSandbox.compile_and_run(harness_source, candidate_source, shared_header)`. The module docstring states plainly that this is prototype-grade isolation (subprocess + sanitizers), not a container or virtual machine.

5.7 backend/app/investigation/ — probe lab and witnesses

`backend/app/investigation/probes.py`

465 lines · Python (generates C)

Name	Purpose
`ProbeError`	Bad scenario; the message is safe to show
`_int(value, name, lo, hi, default)`	Validates one whole number
`world_params(world)`	Returns `world.params`, regenerating it from seed, genes and pins for old stored worlds
`ProbeOutcome`	`ok`, `trace`, `error`, `detail`
`ProbeKit`	Base class: `describe`, `normalize`, `harness`, `parse`, `run`
`_trim_sanitizer(text)`	Keeps the first 12 useful lines of a crash report
`StarvationProbeKit`	`job_queue` experiments (700 jobs, 30 days at most)
`PoolProbeKit`	`pool` experiments (3000 operations at most)
`PROBE_KITS`, `probe_kit(world)`	Look up the kit for a world's archetype

`backend/app/investigation/witnesses.py`

838 lines · Python

Name	Purpose
`_NAMES`	24 first names for witnesses
`_DEFLECT`	Pattern for questions about code, fixes or causes
`Topic`, `WitnessSpec`, `Answer`, `Cast`	Data classes: a question with keywords; a witness; an answer with evidence; the witnesses plus the scenario
`_TraceCache`	In-memory LRU cache of executions (96 entries)
`_route(question, topics)`	Keyword matching from question to topic
`WitnessKit`	Base class: `cast`, `answer`, `lived_period` (cached run), `ask` (routing, deflection, replay), `public` (list for the UI)
`StarvationWitnessKit`	The 7-day load, four users plus the operator, clock formatting (`_clock`, `_adj`, `_dur`), and `_runs` (reconstructs running intervals)
`PoolWitnessKit`	The day of operations, five witnesses, `_physical` (ground reality), and `_said` (how a return value reads in words)
`WITNESS_KITS`, `witness_kit(world)`	Look up the kit for a world

5.8 backend/app/worldgen/ — world generation

`backend/app/worldgen/archetype.py`

120 lines · Python

`GeneratedAgentGrounding` (legacy), `GeneratedWorld`, `DEFAULT_GENES` (all 0.4 except `noise` 0.2), `clamp01`, and the abstract base class `FlawArchetype` with `generate`, `random_pins`, `apply_pins`, `rng` and `resolve_genes`. The module docstring explains the "programs, not content" idea.

`backend/app/worldgen/registry.py`

63 lines · Python

`ARCHETYPE_POOL`, `ARCHETYPES_BY_ID`, `WorldGenerationError`, and `generate_world(session_seed, target_difficulty, archetype_id, engine, max_restarts, pins)`. **To add an archetype, add an instance to `ARCHETYPE_POOL`** (see Chapter 12).

`backend/app/worldgen/evaluate.py`

54 lines · Python

`parse_results(stdout)` reads `VANTA_TEST_RESULT|name|violations|passed` lines. `evaluate_submission(world, source, sandbox)` returns `{"verdict", "reason", "detail", "tests"}` (see the verdict table in section 4.3).

`backend/app/worldgen/archetypes/starvation.py`

373 lines · Python (generates C)

`StarvationArchetype` (`id = "starvation_fairness"`), with `_DOMAINS` (5 settings), the name pools, and `_BUG_VARIANTS`. `generate` draws everything, builds the shared header, requirements, story and params, and

calls `_starter(struct, func, variant)` and `_harness(...)`. The harness contains `allDone`, `runSimulation` (the minute-by-minute simulation with the violation bitmask) and `report`, and runs the 8 tests from section 4.1.

`backend/app/worldgen/archetypes/pool_leak.py`

274 lines · Python (generates C)

`PoolLeakArchetype` (`id = "pool_leak"`), with `_DOMAINS` (4 settings), the name pools, and `_BUG_VARIANTS`. It has `_checkout_impl` (with or without the double-checkout guard), `_starter` (checkout + release for the chosen variant), and `_harness` (`reset`, `countInUse`, `report`, and the 8 tests).

5.9 backend/tests/ — automated tests

Chapter 9 describes every test. In short:

File	Lines	Covers
`backend/tests/test_evolution.py`	80	Viable worlds for both archetypes, determinism, archetype variety, starter rejected and fix accepted under sanitizers
`backend/tests/test_investigation.py`	182	Probe lab behaviour and validation, witnesses (determinism, deflection, routing, replay), ratings, and an HTTP end-to-end test
`backend/tests/test_cohorts.py`	163	Pins, originality, cohort lifecycle, host view, HTTP permissions, starters never explaining their bug
`backend/tests/test_discord_commands.py`	149	Every Discord command against the real API, and the slash-command tree built offline

5.10 discord_bot/ — the Discord adapter

`discord_bot/__init__.py`

1 line

A package marker with a one-line description.

`discord_bot/vanta_api.py`

108 lines · Python · httpx

`VantaApiError(status, detail)`, and `VantaApi(client, admin_token)` with one async method per endpoint the bot uses: `create_world`, `list_worlds`, `get_world`, `submit`, `delete_world`, `delete_my_worlds`, `delete_every_world`, `witnesses`, `ask`, `probe_info`, `probe`, `profile`, `create_cohort`, `get_cohort`, `join_cohort`, `cohort_host`, `close_cohort` and `delete_cohort`. Admin calls add the `X-Vanta-Admin` header.

`discord_bot/commands.py`

370 lines · Python

All command *logic*, with no Discord library involved, so it can be tested.

Name	Purpose
`EmbedSpec`, `Reply`	A description of what to send: text, embeds (title, description, colour, fields, footer, link), files, and whether it is private
`COLOR_*`, `DIFFICULTIES`	The VANTA palette, and Gentle/Standard/Hard/Brutal → 0.2/0.45/0.7/0.9

Name	Purpose
`_cut`, `discord_user`, `difficulty_value`, `extract_source`	Truncation to Discord's limits; `discord:<id>`; calibrated → `None`; strip ```c fences
`VantaCommands`	One method per slash command (`create`, `list`, `info`, `submit`, `delete`, `delete_all`, `admin_delete_all`, `witnesses`, `ask`, `probe`, `profile`, `cohort_create`, `join`, `cohort_status`, `cohort_close`, `cohort_delete`), plus `_world_reply` and `_probe_summary` for formatting

`discord_bot/bot.py`

243 lines · Python · discord.py

The runnable bot. `to_discord(reply)` converts a `Reply` into `discord.Embed` and `discord.File` objects. `_is_admin(interaction)` checks for Manage Server or Administrator. `_read_attachment(att)` reads a `.c` attachment of up to 64 KB. `build_tree(client, cmds)` registers the `/vanta` group with all 16 commands, the choice lists, and autocomplete for witness names. `VantaBot` creates the HTTP client and syncs commands, either to one server (instant) or globally. `main()` reads environment variables and runs the bot.

`discord_bot/requirements.txt`

2 lines

`discord.py>=2.4` and `httpx>=0.27`.

5.11 frontend/ — configuration

File	Lines	Purpose
`frontend/package.json`	28	Scripts (`dev`, `build` = `tsc -b && vite build`, `lint` = oxlint, `preview`) and dependencies (react, react-dom, react-router-dom; dev: vite, tailwindcss, @tailwindcss/vite, typescript, oxlint, type packages)
`frontend/package-lock.json`	2000	Exact dependency versions, generated by npm. Never edit it by hand
`frontend/vite.config.ts`	8	React and Tailwind plugins; the dev-server **proxy** that sends `/api` to `http://127.0.0.1:8811`
`frontend/index.html`	16	The single HTML page: favicon, Google Fonts (Space Grotesk, Inter, JetBrains Mono), `<div id="root">`, loads `src/main.tsx`
`frontend/tsconfig.json`	7	Points to the two configs below
`frontend/tsconfig.app.json`	26	TypeScript settings for `src/`: ES2023, bundler module resolution, `react-jsx`, no unused locals or parameters
`frontend/tsconfig.node.json`	23	TypeScript settings for `vite.config.ts`
`frontend/.oxlintrc.json`	8	Linter rules (React hooks rules, component-export warning)
`frontend/.gitignore`	3	Ignores `node_modules`, `dist` and similar
`frontend/README.md`	32	Vite template readme (scaffold leftover; not VANTA-specific)

5.12 frontend/src/ — application code

`frontend/src/main.tsx`

13 lines · TypeScript (React)

The entry point. It mounts `<App/>` inside `<StrictMode>` and `<BrowserRouter>` into `#root`, and imports the global CSS.

`frontend/src/App.tsx`

44 lines

The page shell: the header (VANTA wordmark, tagline, Guide link, `ProfileBadge`) and the routes `/`, `/guide`, `/worlds/:id`, `/cohorts/:code` and `/cohorts/:code/host`.

`frontend/src/index.css`

82 lines · CSS (Tailwind v4)

Imports Tailwind and defines the design tokens: `--field` (page), `--panel`, `--panel-2`, `--line`, `--line-bright`, `--ink`, `--ink-dim`, `--ink-faint`, `--structure` (cool accent), `--signal` (amber, "look here"), `--accepted`, `--rejected`, and the three font stacks. It also sets up the faint blueprint grid on the page background, focus outlines, and reduced-motion support.

`frontend/src/types.ts`

125 lines

TypeScript interfaces that mirror the API responses: `WorldSummary`, `WorldFull`, `TestResult`, `SubmitResult`, `SubmitResultWithRating`, `RatingChange`, `Archetype`, `ProbeInfo`, `JobRow`, `JobEvent`, `JobTrace`, `PoolOp`, `PoolTrace`, `ProbeResult`, `Witness`, `Evidence`, `WitnessAnswer`, `Profile`, `Cohort`, `CohortMember`, `Originality` and `CohortHostView`.

`frontend/src/api/client.ts`

60 lines

`req(method, path, body)` is a small `fetch` wrapper that throws an `Error` carrying the server's `detail` message. `api` has one function per endpoint: `archetypes`, `create`, `list`, `get`, `submit`, `remove`, `probeInfo`, `probe`, `witnesses`, `ask`, `profile`, `createCohort`, `hostedCohorts`, `cohort`, `joinCohort`, `cohortHost`, `closeCohort` and `deleteCohort`.

`frontend/src/lib/user.ts`

15 lines

`userId()` returns this browser's anonymous id. It is stored in `localStorage` under `vanta_uid`, with an in-memory fallback if storage is blocked. `PROFILE_EVENT` and `notifyProfileChanged()` tell the header badge to refresh.

`frontend/src/components/ui.tsx`

77 lines

Shared building blocks: `Panel` (bordered surface), `Label` (small caps heading), `Chip` (tones: structure, signal, accepted, rejected, neutral), `Button` (primary, ghost, danger) and `Spinner`.

`frontend/src/components/Witnesses.tsx`

190 lines

The Witnesses tab. It shows a list of witness cards and a conversation panel for the selected one. The panel has question chips (grey once asked), a free-text box, and the "also live through the same period under my edited code" checkbox (enabled once the code differs from the starter). Each answer has a "show the record behind this" toggle. `TurnView` shows one question and answer, side by side when there is a replay. `Bubble` shows one answer.

`frontend/src/components/ProbeLab.tsx`

324 lines

The Probe lab tab. It has a JSON editor (pre-filled with the template), help text, a "run it against: the original system / my edited code" switch, and Run and Reset buttons. Results are shown by `JobResult` (stat tiles, an SVG day-by-day timeline with refusal markers, and a per-entry table) or `PoolResult` (stat tiles, the in-use versus receipts chart, the final slot grid, and the operation log). `Stat` and `clock` are helpers.

`frontend/src/components/EvidenceTable.tsx`

43 lines

Renders a witness's evidence (`columns` , `rows` , `total`) as a compact table, highlighting "never started" in amber.

`frontend/src/components/ProfileBadge.tsx`

28 lines

`useProfile()` is a hook that loads `/api/profiles/<uid>` and reloads on `PROFILE_EVENT` . `ProfileBadge` shows the level and rating in the header.

`frontend/src/pages/LandingPage.tsx`

231 lines

The home page: the hero text, the generate panel (`ArchChip` buttons for the defect class and difficulty, including *Calibrated to you* with your recommended value), join-by-code and host-a-cohort panels, the list of cohorts you host, and your open worlds with Open and Delete.

`frontend/src/pages/WorldPage.tsx`

193 lines

One world. It loads the world, keeps the editor text in state, and shows the Investigate tabs (both stay mounted, so switching tabs keeps their state). It submits with the user id, then shows the verdict, check chips, a note about unnamed hidden checks, and the rating change, and finally notifies the badge.

`frontend/src/pages/CohortJoinPage.tsx`

92 lines

Shows the cohort's name, defect class, member count and difficulty. It offers a join form (your display name) or "Open your world" if you are already a member, and links hosts to their console.

`frontend/src/pages/CohortHostPage.tsx`

224 lines

The host console. It refreshes every 8 seconds. It shows the join code panel (copy link, Discord command), the four count tiles, approach groups, most similar pairs with bars (amber at 85% or more), the member table, a hidden-by-default reveal of the planted defect (`VARIANT_LABEL` gives plain-English names), and close and delete with confirmation.

`frontend/src/pages/GuidePage.tsx`

379 lines

The in-app guide for new users (route `/guide`). It explains what VANTA is, your first world in six steps, how witnesses and the probe lab work (with copyable example experiments), checks and ratings, cohorts for participants and hosts, Discord, a FAQ, and a glossary. It has a sticky section menu on wide screens.

Assets and scaffold leftovers

File	Status
`frontend/public/favicon.svg`	The browser-tab icon (currently the default Vite logo)
`frontend/public/icons.svg`	Vite template social icons. **Unused**
`frontend/src/assets/hero.png` , `react.svg` , `vite.svg`	Vite template images. **Unused** (safe to delete)

5.13 docs/ — this documentation

File	Purpose
`docs/src/*.md`	The chapters of this document, written in Markdown with inline SVG diagrams
`docs/build_docs.py`	Builds the PDF: Markdown → HTML (numbering, highlighting, live line counts) → Chromium PDF in two passes for exact page numbers in the contents; merges the cover; adds bookmarks
`docs/render_pdf.cjs`	The Node/Playwright script that prints an HTML file to PDF with page footers and an outline
`docs/VANTA-Technical-Documentation.pdf`	The built document
`docs/build/`	Intermediate files (ignored by git)

`scenarios/` is an empty folder reserved for saved experiment scenarios.

5.14 Generated and local files (not in git)

Path	What it is
`backend/vanta_worlds.db`	The SQLite database (all worlds, ratings, cohorts). Delete it to start fresh
`backend/.venv/`	Python virtual environment (FastAPI, uvicorn, pytest, discord.py, ...)
`backend/server.log` , `frontend/vite.log`	Output of `run_dev.sh`
`frontend/node_modules/`	Installed npm packages
`frontend/dist/`	Production build output of `npm run build`
`__pycache__/` , `.pytest_cache/`	Python caches

CHAPTER 6

HTTP API reference

All endpoints are under `/api` on the backend (default `http://127.0.0.1:8811`), except `/health`. Requests and responses are JSON. Errors use FastAPI's shape `{"detail": "<message>"}` with an HTTP status code.

FastAPI also serves interactive documentation of every endpoint at `http://127.0.0.1:8811/docs` while the server is running.

6.1 Conventions

Item	Rule
Identity	`created_by`, `user_id`, `host_id` and `requested_by` are free-form strings: the browser's random id, or `discord:<id>`
Admin	Send the header `X-Vanta-Admin: <token>` (see section 4.11)
Status codes	200 OK · 400 bad input or generation error · 403 not allowed · 404 not found · 422 invalid scenario, code or body · 500 generation failed
Contestant safety	No endpoint except the admin one returns ground truth, genes, params or reference fixes

6.2 Service and catalogue

`GET /health`

Returns `{"status": "ok"}`. Used to check the server is up.

`GET /api/archetypes`

Lists the defect classes in the pool.

```
[{"id": "starvation_fairness", "title": "Shared Resource Fairness", "domain": "resource_allocation"},
 {"id": "pool_leak", "title": "Resource Pool Integrity", "domain": "resource_pool"}]
```

6.3 Worlds

`POST /api/worlds` — evolve a new world

Body field	Type	Meaning
`created_by`	string, required	Owner id
`target_difficulty`	number 0–1, or `null`	`null` = calibrate to the owner's rating
`archetype_id`	string, optional	Force a defect class; omit for a random one
`seed`	integer, optional	Reproduce a specific world

The response is the contestant view of the world, plus `calibrated`:

```
{
 "world_id": "14fdd16f", "title": "Campus Parking Garage", "archetype_id": "pool_leak",
 "created_by": "docs-demo", "seed": 4242, "target_difficulty": 0.45, "viable": true,
 "briefing": "The campus parking garage manages a fixed number of parking spots that…",
```

```
  "specification": "Implement returnSlot(), which reclaims a parking spot when its holder returns it.
…",
  "starter_source": "int claimSlot(struct Bay pool[], …",
  "function_name": "returnSlot",
  "function_signature": "int returnSlot(struct Bay pool[], int poolSize, int holderId)",
  "mandatory_properties": ["Every parking spot that is released must become available again.", "…"],
  "optional_objectives": ["Minimize the time to find a free slot.", "…"],
  "sample_input_description": "The pool has 12 parking spots. Build your own checkout/release
sequences.",
  "cohort": null,
  "calibrated": false
}
```


Errors: 400 for an unknown `archetype_id`. This call can take from about 1 second (pool) to around 30 seconds (scheduler), because the world is evolved and proven before it is returned.

`GET /api/worlds?created_by=<id>`

Lists world summaries (`world_id`, `title`, `archetype_id`, `created_by`, `seed`, `target_difficulty`, `viable`), newest first. Without `created_by` it lists every world.

`GET /api/worlds/{world_id}`

The contestant view, the same shape as above but without `calibrated`. `cohort` is `{code, name, display_name}` when the world belongs to a cohort member. Returns 404 if the world is unknown.

`POST /api/worlds/{world_id}/submit` — grade a fix

Body: `{"c_source": "<C code>", "user_id": "<id, optional; defaults to the world's creator>"}`.


```
{
  "verdict": "rejected",
  "reason": "failed_tests",
  "detail": null,
  "tests": [
    {"test": "public: basic", "passed": false, "hidden": false},
    {"test": "public: full rejected", "passed": true, "hidden": false},
    {"test": "hidden check 1", "passed": false, "hidden": true},
    {"test": "hidden check 2", "passed": false, "hidden": true}
  ],
  "rating": {"before": 1100.0, "after": 1098.4, "counted": true}
}
```


`reason` is one of `null` (accepted), `failed_tests`, `compile_error`, `timeout` or `runtime_error`. `detail` carries the compiler or timeout message only for `compile_error` and `timeout`. `rating.counted` is `false` once the world has been solved, or after the 5th counted rejection on that world.

`DELETE /api/worlds/{world_id}?requested_by=<id>`

Deletes one world. Allowed if `requested_by` equals the world's creator, or with an admin header. Returns `{"deleted": "<id>"}`, 403 if not allowed, or 404 if not found.

`DELETE /api/worlds?created_by=<id>&requested_by=<id>`

Deletes all of one person's worlds (`requested_by` must match, or admin). Without `created_by` it deletes **every** world and requires admin. Returns `{"deleted_count": N}`.

6.4 Investigation

`GET /api/worlds/{world_id}/probe`

Describes the experiment format for this world:

```
{"kind": "pool", "pool_size": 12, "unit": "parking spot",
 "checkout_func": "claimSlot", "release_func": "returnSlot", "limits": {"ops": 3000},
 "help": "A list of operations run in order on an empty pool of 12 parking spots. …",
 "template": {"ops": [{"op": "checkout", "holder": 1}, {"op": "checkout", "holder": 2},
                      {"op": "release", "holder": 1},
                      {"op": "cycle", "holder": 100, "repeat": 3, "step": 1}]}}
```


For scheduler worlds, `kind` is `job_queue` and the response includes `day_length`, `noun`, `machine` and `limits: {jobs: 700, days: 30}`.

`POST /api/worlds/{world_id}/probe` — run an experiment

Body: `{"scenario": {...}, "c_source": "<C or null>", "user_id": "<id, optional>"}`. When `c_source` is `null` the original starter is run.


```
{"ok": true, "error": null, "detail": null, "ran": "original",
 "trace": {"kind": "pool", "pool_size": 12,
   "ops": [{"k": 0, "op": "checkout", "holder": 5, "returned": 0, "outcome": "got_slot",
            "in_use": 1, "ledger": 1, "entry": 0, "owner": null},
           {"k": 1, "op": "release", "holder": 5, "returned": 0, "outcome": "refused",
            "in_use": 1, "ledger": 1, "entry": 1, "owner": null}],
   "slots": [{"slot": 0, "in_use": true, "holder": 5}, "…"],
   "stats": {"pool_size": 12, "in_use": 1, "ledger": 1, "op_count": 2},
   "ops_truncated": false}}
```


Pool `outcome` values are `got_slot`, `denied`, `invalid_slot`, `accepted` and `refused`. For scheduler traces see section 4.4: `jobs`, `events`, `stats`, `day_length` and `events_truncated`. If the code does not compile, crashes or times out, the response has `ok: false`, `error` set to `compile_error`, `runtime_error` or `timeout`, and `detail`. An invalid scenario returns **422** with a message such as `'jobs[0].minutes' must be between 1 and 20000 (got 0)`.

`GET /api/worlds/{world_id}/witnesses`


```
[{"key": "regular", "name": "Zara",
  "role": "Borrows a parking spot and brings it back, many times a day",
  "vantage": "Sees what the system told them each time they borrowed or returned.",
  "questions": [{"topic": "story", "question": "How did borrowing go for you today?"},
                {"topic": "denied", "question": "Were you ever told there was nothing available?"},
                {"topic": "returns", "question": "Did your returns go through?"}]},
 "… staff, double, newcomer, auditor"]
```


`POST /api/worlds/{world_id}/witnesses/{key}/ask`

Body: `{"topic": "<topic>"}` **or** `{"question": "<free text>"}`, plus optional `c_source` (relive the period under this code) and `user_id`.


```
{"witness": "auditor", "topic": "reconcile",
 "text": "At close: the desk's receipts say 9 out, the system marks 9 as in use, and 8 are physically
out with people. They don't agree.",
 "evidence": {"columns": ["source", "parking spots out"],
              "rows": [["desk receipts", 9], ["system in use", 9], ["physically out", 8]]},
 "followups": ["When did the numbers first stop agreeing?", "What does the end-of-day stock check
show?"]}
```


With `c_source`, the response also has `"replay": {"text": "...", "evidence": {...}}`, or `"replay": {"error": "compile_error", "detail": "...", "text": "..."}`. A question about code or fixes returns `"topic": "deflect"`. An unrecognized question returns `"topic": "unknown"` with `followups`. An unknown witness key returns 404.

6.5 Profiles

`GET /api/profiles/{user_id}`


```
{"user_id": "docs-demo", "rating": 1098.4, "level": "Apprentice", "recommended_difficulty": 0.31,
 "worlds_solved": 0, "worlds_attempted": 1, "submissions": 1,
 "experiments_run": 0, "witness_questions": 0,
 "history": [{"world_id": "14fdd16f", "difficulty": 0.45, "verdict": "rejected", "passed": 3,
              "total": 8, "rating_before": 1100.0, "rating_after": 1098.4,
              "at": "2026-10-05 04:37:05"}]}
```


Unknown users get a fresh profile (rating 1100), not an error.

6.6 Cohorts

`POST /api/cohorts`

Body: `{"host_id", "name", "archetype_id"?, "target_difficulty"? (default 0.45)}`. Returns the public cohort:


```
{"code": "GXDHH6", "name": "CS1 lab · week 6", "archetype_id": "pool_leak",
 "archetype_title": "Resource Pool Integrity", "target_difficulty": 0.45, "status": "open",
 "member_count": 0, "host_id": "k3j9x2ab", "created_at": "2026-10-05 04:17:12"}
```


`GET /api/cohorts?host_id=<id>`

The cohorts this person hosts, each with its `member_count`.

`GET /api/cohorts/{code}?user_id=<id>`

The public cohort, plus `membership` (`{user_id, display_name, world_id, joined_at}` or `null`) when `user_id` is given. Codes are case-insensitive. Returns 404 if unknown.

`POST /api/cohorts/{code}/join`

Body: `{"user_id", "display_name"}`. Returns `{"member": {"user_id", "display_name", "world_id", "rejoined"}, "world": <contestant world view>}`. Returns 400 if the cohort is closed to new members.

`GET /api/cohorts/{code}/host?host_id=<id>`

Host or admin only (otherwise 403). Returns the public cohort plus:

Field	Contents
`setting`	The story place, for example "the university library"
`defect_variant`	The planted variant id (the host UI hides it until revealed)
`defect_class`	The archetype title
`members`	`[{user_id, display_name, world_id, joined_at, world_exists, identifiers, submissions, solved, solved_at, best: {passed, total}, experiments, witness_questions}]`
`solved_count`	Members with an accepted submission
`originality`	The `analyze()` report (section 4.8)

`POST /api/cohorts/{code}/close`

Body: `{"host_id"}`. Host or admin. Sets the status to `closed`.

`DELETE /api/cohorts/{code}?host_id=<id>`

Host or admin. Deletes the cohort and every member's world. Returns `{"deleted": "<CODE>", "worlds_deleted": N}`.

6.7 Admin

`GET /api/admin/worlds/{world_id}`

Admin header required (403 otherwise). Returns the contestant view plus `ground_truth_summary` (for example `archetype=pool_leak bug_variant=off_by_one_index pool_size=10 …`), `genes` and `fitness` (`broken`, `fixable`, `distinct_fixes`, `difficulty_error`, `score`, `generation`).

CHAPTER 7

Data model

All persistent state lives in **one SQLite file**, `backend/vanta_worlds.db`. Each store creates its own tables the first time it connects (`CREATE TABLE IF NOT EXISTS`), so there is no migration step. Deleting the file resets VANTA completely.

7.1 What "nothing is stored" means

VANTA stores *instances*, not *content*. No problem text, answer key, bug list or witness script is written in advance anywhere. Once a world has been generated for someone, its generated text and code are saved so they can come back to it. Ratings, cohorts and submissions are saved too. Witness statements and experiment results are never saved: they are recomputed by running code, and only cached in memory for speed.

7.2 Table `worlds`

Created by `world_manager._connect()`.

Column	Type	Meaning
`world_id`	TEXT, primary key	8 characters from a UUID
`seed`	INTEGER	The seed requested for this world
`archetype_id`	TEXT	`starvation_fairness` or `pool_leak`
`title`	TEXT	Story title
`created_by`	TEXT	Owner id
`target_difficulty`	REAL	0–1, used for ratings
`viable`	INTEGER (0/1)	Whether evolution proved it viable
`world_json`	TEXT	The serialized `GeneratedWorld` (see below)
`fitness_json`	TEXT	`broken`, `fixable`, `distinct_fixes`, `difficulty_error`, `score`, `generation`
`created_at`	TEXT	UTC timestamp, set by SQLite

`world_json` holds `archetype_id`, `seed` (the winning candidate's own seed), `genes`, `title`, `briefing`, `specification`, `starter_source`, `function_name`, `function_signature`, `shared_header`, `mandatory_properties`, `optional_objectives`, `sample_input_description`, `harness_source`, `ground_truth_summary` and `params`.

`params` contains the struct and function names, the variant, the constants, the `domain` dictionary, `domain_index` and `pins`. Worlds saved before `params` existed are handled by `world_params()`, which regenerates the params from the seed and genes.

7.3 Table `profiles`

Created by `profiles._connect()`.

Column	Type	Meaning
`user_id`	TEXT, primary key	Person id
`rating`	REAL	Current Elo-style rating
`updated_at`	TEXT	Last change

7.4 Table `attempts`

One row per submission, used for history and the anti-farming rules.

Column	Type	Meaning
`id`	INTEGER, autoincrement	
`user_id`, `world_id`	TEXT	Who submitted, and to which world
`difficulty`	REAL	The world's target difficulty
`verdict`	TEXT	`accepted` or `rejected`
`passed`, `total`	INTEGER	Checks passed, and checks in total
`rating_before`, `rating_after`	REAL	The rating change (equal when it didn't count)
`created_at`	TEXT	Timestamp

7.5 Table `activity`

Investigation counters. This table is created by both the profiles store and the cohort store.

Column	Type	Meaning
`id`	INTEGER	
`user_id`, `world_id`	TEXT	
`kind`	TEXT	`probe`, `witness` or `replay`
`created_at`	TEXT	

7.6 Table `cohorts`

Column	Type	Meaning
`cohort_id`	TEXT, primary key	8 characters
`code`	TEXT, unique	6-character join code (uppercase)
`name`	TEXT	Up to 80 characters
`host_id`	TEXT	Who created it
`archetype_id`	TEXT	The defect class for everyone
`pins_json`	TEXT	`{"variant": "...", "domain": N}`

Column	Type	Meaning
`target_difficulty`	REAL	One difficulty for everyone
`seed`	INTEGER	Cohort seed (31 random bits); member seeds are derived from it
`status`	TEXT	`open` or `closed`
`created_at`	TEXT	

7.7 Table `cohort_members`

Primary key `(cohort_id, user_id)`, so each person joins a given cohort once.

Column	Type	Meaning
`cohort_id`, `user_id`	TEXT	
`display_name`	TEXT	Up to 40 characters, shown to the host
`world_id`	TEXT	Their personal world
`signature`	TEXT	Their identifier set, for example `Lease|leaseSlot|freeSpot`, kept unique within the cohort
`joined_at`	TEXT	

7.8 Table `cohort_submissions`

Every submission to a cohort world, *including the source code*, for the originality analysis.

Column	Type	Meaning
`id`	INTEGER	
`cohort_id`, `user_id`, `world_id`	TEXT	
`verdict`	TEXT	
`passed`, `total`	INTEGER	
`source`	TEXT	The submitted C code
`created_at`	TEXT	

7.9 How the tables relate

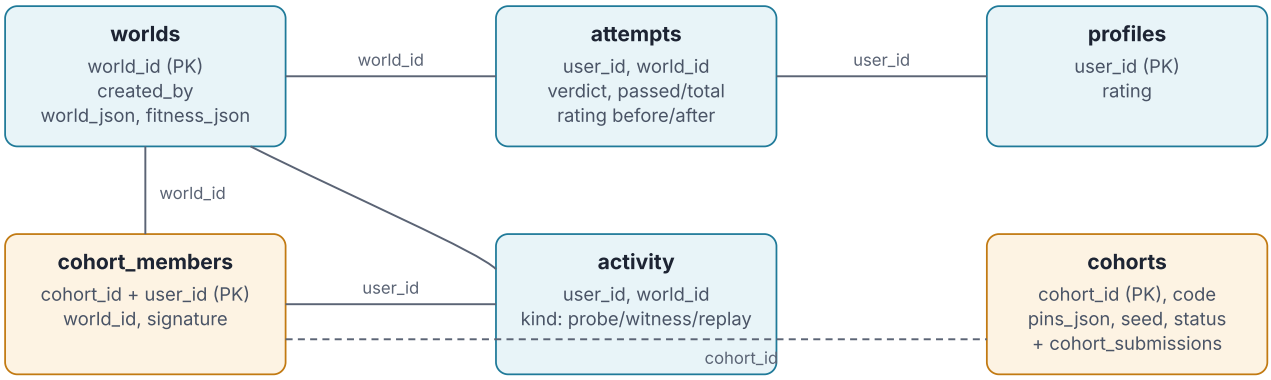


Figure 7.1. Logical links between tables. SQLite foreign keys are not declared; the links are kept by the code.

7.10 What is kept only in memory

Data	Where	Lifetime
Witness execution traces	`witnesses._cache` (LRU, 96 entries)	Until the server restarts
The browser's user id	`localStorage["vanta_uid"]`	Until browser storage is cleared
Witness conversations and probe results on screen	React state	Until the page is closed

CHAPTER 8

C protocols and formats

VANTA's Python side and its generated C programs talk through plain text printed to standard output. This chapter defines those formats exactly.

8.1 How the two C files are put together

```
harness_main.c = shared_header + "\n" + harness_source      (has main())
candidate.c    = shared_header + "\n" + contestant source   (has the functions)
gcc harness_main.c candidate.c -o a.out
```

Each file is a separate *translation unit*, so both get the shared header (struct + prototypes). The contestant must define every function that the shared header declares and the harness calls. In pool worlds that means both the checkout and the release function.

8.2 Grading output

Printed by every hidden-test harness:

```
VANTA_TEST_RESULT|<test name>|<violations>|<passed 0/1>
...
VANTA_OVERALL|<1 if every test passed, else 0>
```

- `violations` is a number. For scheduler worlds it is a bitmask: 1 = a job was starved, 4 = a job was started that could not finish before closing, 16 = a bad pick. For pool worlds it is 0 for a pass and 1 for a fail.
- The program exits with 0 if everything passed and 1 otherwise.
- If `VANTA_OVERALL|` never appears, the program crashed (a sanitizer trap, a segmentation fault, or an `abort`), and the verdict is `runtime_error`.

8.3 Scheduler probe output (`job_queue`)

```
PROBE_EVENT|<minute>|<kind>|<job id or index>        (at most event_cap lines)
PROBE_JOB|<id>|<minutes>|<submitted>|<started or -1>|<finished or -1>|<status>
PROBE_STAT|<busy>|<open>|<idle_waiting>|<too_late>|<invalid>|<days>|<events>|<max_queue>
PROBE_DONE
```

Event kind	Meaning
`start`, `finish`	The machine began or completed a job
`abandoned`	The owner walked away (`abandon_after`)
`freed`	A running job was abandoned, so the machine stopped it
`rejected`	The job was longer than a whole day, so it was refused up front
`refused_too_late`	The scheduler chose a job that couldn't finish before closing; the machine idled that minute (logged once per job per day)
`invalid_pick`	The scheduler chose a job not yet submitted, already finished, or already started
`invalid_index`	The scheduler returned an index outside 0...count−1

Job ids are assigned 1, 2, 3 … in the order the scenario entries are expanded. `PROBE_DONE` is always the last line. If it is missing, the run crashed.

8.4 Pool probe output (`pool`)

```
PROBE_OP|<k>|<0 checkout / 1 release>|<holder>|<returned>|<in use after>|<receipts after>
PROBE_SLOT|<slot>|<in use 0/1>|<holderId stored in memory>
PROBE_STAT|<pool size>|<in use>|<receipts>|<operation count>
PROBE_DONE
```

"Receipts" are kept by the harness: +1 when a checkout returns something other than −1, and −1 when a release returns non-zero.

8.5 Scenario JSON schemas

Scheduler

```
{
  "days": int 1..30            (optional, default 3)
  "jobs": [                    (required, 1..700 jobs after expansion)
    {
      "minutes": int 1..20000,             required
      "at": int 0..days*L,                 required, submission minute
      "repeat": int 1..700,                optional, default 1
      "every": int 1..100000,              optional, default 1
      "abandon_after": int 0..1000000,     optional, minutes after submission
      "owner": string                      optional, used internally by witnesses
    }
  ]
}
```

Pool

```
{
  "ops": [                     (required, 1..3000 operations after expansion)
    {
      "op": "checkout" | "release" | "cycle",   required
      "holder": int 0..10000000,                required
      "repeat": int 1..3000,                    optional, default 1
      "step": int 0..100000,                    optional, default 0
      "owner": string                           optional
    }
  ]
}
```

Booleans and non-integer numbers are rejected. Values out of range produce a 422 error that names the field, for example `'ops[2].holder' must be between 0 and 10000000 (got -4)`.

8.6 Code safety rules

Before anything is compiled, contestant code (for grading, probes and replays) is checked against these patterns. A match is reported as a compile error:

```
system(   exec*(   fork(   popen(   fopen(   freopen(   remove(   unlink(   rename(
#include <unistd.h>   #include <sys/...>   #include <fcntl.h>   asm   __asm__
```

Then the code is compiled with sanitizers and run with a 15-second limit (grading and probes) in a temporary folder that is deleted afterwards. Chapter 11 discusses what this does and does not protect against.

CHAPTER 9

Testing

9.1 Running the tests

```
cd backend
.venv/bin/python3 -m pytest tests -q          # all 35 tests, about 4 minutes
.venv/bin/python3 -m pytest tests/test_cohorts.py -q    # one file, about 10 seconds
```

The evolution tests are the slow ones, because they really compile and run dozens of C programs. Tests that touch the database use a `tmp_db` fixture, which points `DB_PATH` at a temporary file, so your real `vanta_worlds.db` is never modified. HTTP tests use `httpx.ASGITransport`, which calls the FastAPI app in-process without opening a network port.

9.2 `test_evolution.py` — the generator

Test	What it proves
`test_pool_leak_evolves_viable_world`	Evolution finds a pool world that is broken, fixable, and has at least 2 distinct fixes
`test_starvation_evolves_viable_world`	The same for a scheduler world
`test_same_seed_reproduces_same_world`	Evolution is deterministic: the same seed gives the same starter and function name
`test_registry_picks_archetype_by_seed`	Over seeds 1–7, both defect classes are chosen, so the class itself varies
`test_starter_rejected_correct_fix_accepted_pool`	Under the *sanitized* sandbox, the starter is rejected and a hand-written correct fix is accepted

9.3 `test_investigation.py` — probe lab, witnesses, ratings

Test	What it proves
`test_probe_reveals_starvation_and_fix_relieves_it`	In a `pure_shortest_first` world, an experiment shows the long job waiting far longer on the original than on a fix
`test_probe_on_pool_shows_double_release_difference`	In a `linear_search_first_match` world, a double release is "accepted" by the original and "refused" by a fix
`test_probe_rejects_bad_scenarios` (5 cases)	Empty lists, code-injection strings, negative times, too many jobs, and non-objects are all refused
`test_probe_reports_compile_errors_in_constant_code`	Broken code gives `ok: false, error: compile_error`, not a crash
`test_every_witness_answers_every_question_deterministically` (2 cases)	For both archetypes, every witness answers every topic, with identical text each time
`test_witness_deflects_requests_for_the_solution`	"How do I fix the code?" is deflected; nonsense gets "unknown" plus suggestions
`test_free_text_routes_to_a_topic`	"How long did you have to wait?" routes to the *worst* topic

Test	What it proves
`test_auditor_books_disagree_originally_and_agree_under_a_fix` (3 cases)	For every pool variant, the auditor finds a mismatch in the original and none under a fix
`test_replay_with_broken_code_is_reported_not_raised`	A replay with code that won't compile returns `replay.error`, not a server error
`test_rating_moves_with_results`	A rejection lowers the rating, an accept raises it, a second accept doesn't count, and the recommended difficulty rises
`test_recommended_difficulty_spans_beginner_to_expert`	Recommendations stay within 0.10–0.95 and rise with rating
`test_investigation_api_end_to_end`	Over HTTP: calibrated create → probe → bad probe gives 422 → witnesses → ask → submit → profile counts

9.4 `test_cohorts.py` — cohorts, originality, permissions

Test	What it proves
`test_pinned_worlds_share_defect_but_not_names`	Two pinned worlds share the variant and setting but not their identifiers
`test_originality_sees_through_renaming_and_separates_approaches`	The same approach in two worlds with renamed variables scores above 0.85; forward and backward scans score below 0.55; the report finds 2 approaches and flags the unchanged starter
`test_cohort_gives_each_member_a_unique_world_with_the_same_defect`	Three members get the same variant and setting, three different identifier sets and three different starters; rejoin returns the same world; only the host sees the console; closing blocks newcomers but not members
`test_host_view_tracks_progress_and_approaches`	The host view counts solved members, 2 distinct approaches, and the unchanged submission; delete removes 3 worlds
`test_cohort_api_and_world_ownership`	Over HTTP: cohort lifecycle, cohort link on the world, 403 for non-hosts, 403 for deleting someone else's world, admin-only "delete everything", admin-only ground truth
`test_starter_code_never_explains_its_own_bug`	No starter (any archetype, any variant) contains a comment or a `(void)` hint

9.5 `test_discord_commands.py` — the Discord bot

Test	What it proves
`test_extract_source_handles_fences`	Code pasted inside ```c fences is unwrapped
`test_world_lifecycle_from_discord`	`create` → `list` → `submit` (rejected, with hidden checks unnamed) → `submit` a fix (accepted, rating shown) → others can't delete → non-admins can't delete everything → owner deletes
`test_witnesses_and_probe_from_discord`	`witnesses`, `ask` (including an unknown name), `probe` help, a probe run, invalid JSON, `profile`, `delete_all`
`test_cohort_from_discord`	`cohort_create`, two `join`s with different starter files, rejoin, a solve, `cohort_status` for host and non-host, `cohort_close`, a late join refused, `cohort_delete`

Test	What it proves
`test_slash_command_tree_builds_offline`	All 16 `/vanta` commands register and serialize correctly with discord.py, without connecting to Discord

9.6 Browser checks

Browser behaviour was verified with scripted headless Chromium (Playwright):

- generating a world, asking witnesses, opening evidence, running the probe lab, and submitting;
- a pool world fixed in the editor, with a witness replay showing "All three agree" and an accepted submission that moves the rating;
- a three-person cohort created from the landing page, joined from three separate browsers, with two different accepted fixes and one rejected starter. The host console showed 2 solved and 2 distinct approaches.

These scripts are not part of the automated suite, because they need both servers running.

9.7 Frontend type-checking

```
cd frontend && npx tsc -b      # type errors fail here
npm run build                  # type-check + production bundle in dist/
```

CHAPTER 10

Installing and running

10.1 Requirements

Tool	Version used	Needed for
Python	3.13 (3.11+ should work)	Backend and Discord bot
gcc	any recent version with AddressSanitizer and UBSan	Compiling worlds and submissions
Node.js + npm	Node 22	Frontend
A modern browser	—	Using the web UI

The sanitizers need a Linux or macOS toolchain. On Windows, use WSL (Windows Subsystem for Linux).

10.2 First-time setup

```
# 1. backend
cd backend
python3 -m venv .venv
.venv/bin/pip install fastapi uvicorn pydantic httpx pytest pytest-asyncio "discord.py>=2.4"

# 2. frontend
cd ../frontend
npm install
```

10.3 Starting everything

```
./run_dev.sh            # from the repository root: API on :8811, web on :5173
```

Or start each one in its own terminal:

```
cd backend  && .venv/bin/python3 -m uvicorn main:app --host 127.0.0.1 --port 8811
cd frontend && npm run dev -- --host 127.0.0.1 --port 5173
```

Then open `http://127.0.0.1:5173`. The Vite dev server forwards `/api` to port 8811. The interactive API explorer is at `http://127.0.0.1:8811/docs`.

Production build. `npm run build` writes static files to `frontend/dist/`. Serve them with any web server that forwards `/api` to the backend.

10.4 Configuration (environment variables)

Variable	Read by	Meaning
`VANTA_ADMIN_TOKEN`	API and bot	Secret for admin actions. If unset on the API, any `X-Vanta-Admin` header counts as admin (demo mode)
`DISCORD_TOKEN`	bot	The bot's token from the Discord Developer Portal (required for the bot)
`VANTA_API_URL`	bot	Where the API is (default `http://127.0.0.1:8811`)
`VANTA_WEB_URL`	bot	If set, world embeds link to `<url>/worlds/<id>`

Variable	Read by	Meaning
`DISCORD_GUILD_ID`	bot	Sync commands to this one server instantly (otherwise global sync, which can take up to an hour)

10.5 Setting up the Discord bot, step by step

1. Go to the Discord Developer Portal, create an **Application**, open **Bot**, and copy the **token**.
2. Under **OAuth2 → URL Generator** choose the scopes `bot` and `applications.commands` and the permissions *Send Messages*, *Attach Files* and *Embed Links*. Open the generated URL and add the bot to your server.
3. Turn on Discord's Developer Mode, right-click your server, and choose *Copy Server ID*.
4. With the API running, start the bot from the repository root:

```
pip install -r discord_bot/requirements.txt
DISCORD_TOKEN=... DISCORD_GUILD_ID=... \
VANTA_API_URL=http://127.0.0.1:8811 VANTA_WEB_URL=http://127.0.0.1:5173 \
python -m discord_bot.bot
```

Step 5. Type `/vanta` in your server. The 16 commands should appear.

For admin commands to work with a token configured, set the **same** `VANTA_ADMIN_TOKEN` for the API and the bot.

10.6 Ports and processes

Process	Port	Log (when started by run_dev.sh)
Uvicorn (FastAPI)	8811	`backend/server.log`
Vite dev server	5173	`frontend/vite.log`
Discord bot	— (outbound only)	console

10.7 Common tasks

Task	How
Reset everything	Stop the API, delete `backend/vanta_worlds.db`, start it again
Delete every world (keep ratings and cohorts)	`curl -X DELETE -H "X-Vanta-Admin: <token>" http://127.0.0.1:8811/api/worlds`
See a world's planted bug	`curl -H "X-Vanta-Admin: <token>" http://127.0.0.1:8811/api/admin/worlds/<id>`
Rebuild this PDF	`python3 docs/build_docs.py` (needs `markdown`, `pygments`, `pypdf`, Node and Playwright)
Run the tests	`cd backend && .venv/bin/python3 -m pytest tests -q`

CHAPTER 11

Limitations, security and future work

A good project document is honest about what is not finished. Everything here is a known, deliberate limit of the current prototype.

11.1 Security limits

The C sandbox is prototype-grade.

Contestant code runs as a normal process on the server, after a pattern check and with sanitizers and a timeout. A determined attacker could find calls that aren't on the deny list. Examples are raw `syscall()`, other ways of reaching the filesystem, or using up memory. For a public deployment, run the sandbox inside a container or VM (for example Docker with no network, a read-only filesystem, and memory and CPU limits, or nsjail or gVisor). `compile_and_run` is the only function that would need to change.

Area	Limit	What to do for real use
Identity	Browser ids live in `localStorage` and Discord ids are trusted; there are no passwords. Anyone who knows a world id can submit to it	Add real accounts (OAuth or university sign-in) and attach worlds to accounts
Admin	Demo mode (no `VANTA_ADMIN_TOKEN`) treats any `X-Vanta-Admin` header as admin	Always set the token outside a demo
Host rights	Proven only by knowing the host's id	The same fix as identity
Rate limits	None; generating worlds and probing cost CPU	Add per-user limits and a job queue
CORS	Allows every origin	Restrict to the deployed frontend's origin

11.2 Functional limits

- **Two defect classes.** The pool has `starvation_fairness` and `pool_leak`, with three variants each. More archetypes (Chapter 12) would broaden it, for example integer overflow in billing, a ring buffer, an LRU cache, string parsing, or a state machine.
- **Some genes are reserved.** `system_size`, `noise` and `reasoning_depth` are produced and evolved, but the current archetypes don't read them yet.
- **Difficulty is estimated, not measured on people.** The estimate uses genes and how obvious the starter's failures are. Real solve rates could be fed back to improve it.
- **Witness wording is templated.** Answers are computed from executions and phrased with fixed sentence templates. No language model is involved, which keeps them exact. A future option is an LLM that only rephrases the computed facts.
- **Single process, SQLite.** This is fine for a class or a hackathon room. Many simultaneous users would need a server database and background workers for evolution.
- **Discord not tested live.** The command logic is tested against the real API, and the command tree is built and serialized. The bot has not been run against Discord's servers from the development environment, because that needs a bot token.

- **The originality analysis measures diversity, not cheating.** Two honest people can make the same one-line fix. The host console says so.
- **Leaderboard.** Deliberately not built yet.
- **Scaffold leftovers.** The default Vite favicon and a few unused template images remain in `frontend/`.

11.3 Ideas for future work

1. More archetypes, and per-archetype difficulty tuned from real solve data.
2. A container-based sandbox.
3. Accounts and a hackathon layer: rounds, time limits, and optionally a leaderboard.
4. Saving and sharing experiments (the `scenarios/` folder is reserved for this).
5. Letting the host pick which witnesses appear, or adding "decoy" witnesses driven by the `noise` gene.
6. Exporting a cohort's results (per member and per approach) as a spreadsheet.

CHAPTER 12

Extending VANTA: adding a defect class

The most valuable way to grow VANTA is to add archetypes. Each new archetype automatically gets evolution, grading, ratings, cohorts, originality analysis, the web UI and Discord. Only the probe lab and witnesses need archetype-specific code. This chapter is a checklist.

12.1 1. Write the archetype

Create `backend/app/worldgen/archetypes/<name>.py`:

```python
from app.worldgen.archetype import FlawArchetype, GeneratedWorld

_DOMAINS = [{"noun": "...", "place": "..."}, ...]
_STRUCT_NAMES = [...]              # wide pools: cohorts need unique names
_BUG_VARIANTS = ["variant_a", "variant_b", "variant_c"]

class RingBufferArchetype(FlawArchetype):
    id = "ring_buffer"
    title = "Message Buffer Integrity"
    domain = "data_structures"
    variants = _BUG_VARIANTS
    domain_count = len(_DOMAINS)

    def generate(self, seed, genes=None, pins=None) -> GeneratedWorld:
        genes = self.resolve_genes(genes)
        rng = self.rng(seed)
        domain_index = rng.randrange(len(_DOMAINS))
        struct_name = rng.choice(_STRUCT_NAMES)
        variant = rng.choice(_BUG_VARIANTS)
        variant, domain_index = self.apply_pins(pins, variant, domain_index)
        # ... draw constants from rng, scaled by genes ...
        # ... build shared_header, starter_source (NO comments that explain the bug),
        #     harness_source (prints VANTA_TEST_RESULT / VANTA_OVERALL lines) ...
        return GeneratedWorld(..., params={"struct_name": struct_name, "func_name": ...,
                                           "bug_variant": variant, "domain": _DOMAINS[domain_index],
                                           "domain_index": domain_index, "pins": pins or None})
```

Rules that keep it correct (each one was learned the hard way in this project):

- **Every random choice must come from `rng`**, so a seed reproduces exactly.
- **The harness must exercise the real defect.** For example, a starvation test that lets the queue drain never starves anyone, so the bug is never triggered.
- **Every hidden test must check the full set of required properties**, not a subset. A narrow check let broken variants pass.
- **No hints in the starter.** No comments, and no `(void)param;` lines. A test enforces this.
- **Use `params` for every name and constant** that the probe lab, witnesses or originality analysis will need.

12.2 2. Add two different reference fixes

In `backend/app/evolution/reference_fixes.py`, add at least two functions `(world) -> str` that solve the problem *in different ways*, and register them:

```python
REFERENCE_FIXES["ring_buffer"] = [_ring_modulo_indices, _ring_count_field]
```

Without at least two passing fixes, no world can be viable.

12.3 3. Register it

Add an instance to `ARCHETYPE_POOL` in `backend/app/worldgen/registry.py`. It now appears in `/api/archetypes`, in the landing page chips, and in random picks. The Discord `defect` choices are a fixed list in `discord_bot/bot.py` (`DEFECT_CHOICES`), so add it there too.

12.4 4. Probe kit (experiments)

Subclass `ProbeKit` in `backend/app/investigation/probes.py`:

- `describe` returns help text, limits and a template;
- `normalize` validates every field with `_int` and expands repeats;
- `harness` writes a C `main()` that only uses validated integers and ends with `PROBE_DONE`;
- `parse` reads your `PROBE_...` lines into a trace.

Add it to `PROBE_KITS`. The frontend `ProbeLab` renders `job_queue` and `pool` traces. A new `kind` needs a result component next to `JobResult` and `PoolResult`.

12.5 5. Witness kit

Subclass `WitnessKit` in `backend/app/investigation/witnesses.py`:

- `cast(world)` returns the witnesses (each with topics and keywords) and one busy-period scenario whose entries carry an `owner`;
- `answer(world, cast, witness, topic, trace)` computes the text and evidence from that witness's slice of the trace only.

Add it to `WITNESS_KITS`.

12.6 6. Tests

Copy the patterns in `backend/tests/`:

- the archetype evolves a viable world;
- every pinned variant evolves a viable world;
- the starter is rejected and a fix is accepted under the sanitized sandbox;
- every witness answers every topic deterministically;
- a probe shows a difference between the original and a fix;
- `test_starter_code_never_explains_its_own_bug` covers it automatically.

12.7 Adding an API endpoint or page

- **Endpoint:** add a function with `@router.get/post/delete` in `backend/app/api/routes.py`. Put logic in a `core/` or `investigation/` module, not in the route. Add a method to `frontend/src/api/client.ts` and a type to `frontend/src/types.ts`.
- **Page:** add a component in `frontend/src/pages/` and a `<Route>` in `frontend/src/App.tsx`. Reuse `Panel`, `Label`, `Chip` and `Button` from `components/ui.tsx`, and the CSS tokens from `index.css`, so it matches the rest of the UI.
- **Discord command:** add a method to `VantaCommands` (returning a `Reply`), wire it in `build_tree`, and add a test in `test_discord_commands.py`.

CHAPTER 13

Presenting VANTA

A script for showing VANTA to a professor or judges in about ten minutes, with the points to make at each step and answers to the questions they are likely to ask.

13.1 Before you start

- Run `./run_dev.sh`, open `http://127.0.0.1:5173`, and generate one world beforehand so the page is warm.
- Open a second browser (or a private window) to play a cohort member.
- Optionally, run `python3 -m pytest tests -q` beforehand and keep the "35 passed" result on screen.

13.2 The script

1. **The pitch (30 seconds).** "Most challenge platforms hand you a stored problem with a stored answer. VANTA writes a broken system on demand, proves it is broken and fixable in more than one way, and grades you by running your C."
2. **Generate (1 minute).** Choose *Calibrated to you* and *Generate a world*. While it spins, explain that it is *evolving*: six candidates, each compiled and tested against its own hidden tests and against two reference fixes, until one is viable.
3. **Read (1 minute).** Show the story, the requirements and the starter. Point out that the names and numbers are unique to this seed, and that the code contains no hint comments.
4. **Witnesses (2 minutes).** Ask the long-job user *"What was the worst wait you had?"*, then open "show the record behind this". Then type *"How do I fix the code?"* to show the deflection. Make the key point: "Nobody holds facts. Everything they say came from running this exact code. There is nothing to argue with."
5. **Probe lab (2 minutes).** Write a small experiment: a stream of short jobs plus one long job. Run it against the original, and show the timeline, the never-started count or the refusal markers.
6. **Fix and replay (2 minutes).** Edit the code (for example, oldest-first among jobs that fit), run the same experiment against *my edited code*, and then tick "live through the same period under my code". Show that the long-job user is now happy and the short-job user waits longer. Make the point: "Two correct designs, different trade-offs."
7. **Submit (30 seconds).** Show *Accepted*, the checks with hidden ones unnamed, and the rating change. Then show that the landing page's calibrated difficulty has moved up.
8. **Cohort (2 minutes).** Host a cohort and join it from the second browser. Show the host console with different identifiers per member and, after two different fixes, *distinct approaches = 2*.

13.3 Likely questions

Question	Answer
"Is this using AI to grade?"	No. Grading is compile + run + read the hidden tests' output. No language model is involved anywhere in grading or in witness answers.
"Where are the problems stored?"	Nowhere. Archetypes are programs. A world exists only after it is generated from a seed, and it is proven by execution.

Question	Answer
"Can students share answers?"	In a cohort, every member's struct and function names differ and the constants differ. Copied code doesn't compile, and the originality analysis groups similar solutions.
"How do you know a world is solvable?"	Evolution only serves worlds where at least two architecturally different reference fixes pass the hidden tests.
"How does it adapt to beginners and experts?"	An Elo-style rating per person. The next world's target difficulty is set just above that rating, and evolution searches for a world near that target.
"Why is this hard to copy?"	The value isn't in any one world. It is in the archetypes, the fitness function, the generate–test–select loop, and the execution-grounded investigation layer.
"What's unsafe about running student C?"	It's checked for dangerous calls and sanitized, but it isn't containerized yet (Chapter 11). For a public deployment, the sandbox would move into a container.
"What would you add next?"	More defect classes, a container sandbox, accounts, and a hackathon round structure.

CHAPTER 14

Glossary

Term	Meaning
Admin header	`X-Vanta-Admin`; grants admin rights when it matches `VANTA_ADMIN_TOKEN` (any value in demo mode)
AddressSanitizer (ASan)	A gcc feature that makes out-of-bounds or invalid memory access crash immediately, instead of silently misbehaving
Approach	A group of accepted cohort solutions whose novel code is at least 55% similar
Archetype	A Python class that generates worlds of one defect class
ASGI	The interface between Python web frameworks and servers; FastAPI is an ASGI app and Uvicorn is an ASGI server
Bitmask	An integer whose individual bits mean different things (1, 4, 16 in scheduler tests)
Busy period	The generated week (or day) of realistic usage that witnesses live through
Calibrated	Difficulty chosen from your rating instead of picked by hand
Cast	The witnesses of a world, plus their busy period
Check	One hidden or public test, as shown to a contestant
Cohort	A group session: same defect and setting for everyone, a unique world each
CORS	Browser rule about which websites may call an API; the backend allows all
Deflection	A witness's polite refusal to discuss code or fixes
Deterministic	The same inputs always give the same output
Distinct fixes	How many architecturally different reference fixes pass a world
Domain / setting	The story frame: which machine or pool, and which place
Elo rating	A rating system from chess; here, worlds are the opponents
Embed	A formatted Discord message card
Evidence / the record	The table of raw execution data behind a witness answer
Evolution	The generate–test–select–mutate loop that finds viable worlds
FastAPI	The Python web framework used for the backend
Fingerprint	A hash of 6 consecutive normalized tokens, used for comparing code
Fitness	The score that says how good a candidate world is
Genes	Six 0–1 difficulty dials
Ground truth	The planted variant and constants; admin-only
Harness	The generated C `main()` that tests the contestant's functions
Hidden check	A test whose name and scenario are not shown to contestants
Jaccard similarity	The size of the intersection divided by the size of the union of two sets
Join code	The 6-character code for a cohort

Term	Meaning
Ledger / receipts	Checkouts granted minus returns accepted, as recorded by the desk
LRU cache	A cache that discards the least recently used entry when full
Mutation	A small random change to genes between generations
Normalize	Rewrite code so names, variables and numbers don't affect comparison
Novel fingerprints	A submission's fingerprints minus its own starter's
Params	Structured names and constants of a world, for internal tools
Physical reality	In pool worlds, who actually holds an item, worked out from what people did
Pins	A fixed variant and setting (used by cohorts)
Probe / experiment	A contestant-designed scenario run against the system
Probe kit	The per-archetype code that validates, builds, runs and parses experiments
Reference fix	A built-in correct solution used only to prove fixability
Replay	A witness living the same busy period again under the contestant's code
Sanitizers	ASan + UBSan, enabled for contestant code
Seed	The integer every random choice in a world comes from
Shared header	The struct and prototypes prepended to both C files
Signature (cohort)	A member's identifier set, kept unique within a cohort
Slash command	A Discord command such as `/vanta create`
SQLite	A database stored in one file, `backend/vanta_worlds.db`
Starter source	The broken code a contestant starts from
Starvation	A request waiting forever because others keep being chosen first
Topic	One question a witness can answer, with keywords for routing free text
Trace	The structured result of running an experiment or busy period
Translation unit	One `.c` file as the compiler sees it; why both files need the header
UBSan	UndefinedBehaviorSanitizer: makes undefined behaviour (such as signed overflow) crash
Variant	Which specific mistake an archetype planted
Viable	Broken, fixable, and at least 2 distinct fixes pass
Vite	The frontend build tool and development server
Witness	A vantage point on one real run of the system, answering bounded questions
World	One generated challenge: story, code, requirements, hidden tests